REACT · LECTURE SERIES · PART 1

Foundations

A complete, in-depth, simple-English guide to the modern JavaScript and tooling you need before React — let / const, arrow functions, destructuring, spread, array methods, async / await & Vite.

Modern JavaScript • Async code • Project setup



LECTURE ROADMAP

What We'll Cover Today

LC

let & const

Declare variables the safe way

=>

Arrow Functions

Short, modern function syntax

${}

Template Literals

Strings that hold variables

{ }

Destructuring

Unpack objects and arrays

...

Spread & Rest

Copy, merge and collect

MAP

Array Methods

map, filter, reduce and more

?.

Optional Chaining & ??

Read deep values safely

IM

Modules

Share code with import / export

AW

Promises & async / await

Wait for slow work cleanly

V

Vite & npm Tooling

Create, run and build the app

THE BIG IDEA

What Exactly Is React?

React is a JavaScript library for building user interfaces out of small, reusable pieces called components. You describe WHAT the screen should look like for the current data, and React works out HOW to update the page.

> IN ONE LINE:
>
> ***"Describe the UI for your data, and React keeps the screen in sync."***

Why developers choose React:

- Declarative: you describe the result, not every step
- Component-based: build once, reuse everywhere
- Virtual DOM: React updates only what changed
- Huge ecosystem: routing, state, UI kits

BEFORE — Plain JavaScript

```
const p = document.createElement('p');
p.textContent = count;
document.body.append(p);

// count changed? update by hand
p.textContent = count + 1;
```

AFTER — React Component

```
function Count({ count }) {
  return <p>{count}</p>;
}

// count changed?
// React updates the screen
```

BEFORE YOU START

The Two Golden Rules of Modern JS for React

1 Never change data directly

Treat objects and arrays as read-only. Make a new copy with spread, map or filter and hand the copy to React, so it can see that something changed.

DO

```
const next = [...items, newItem];
setItems(next);
```

DON'T

```
items.push(newItem);
setItems(items); // same array!
```

2 Keep functions predictable

Same inputs should give the same output, and the function should not change anything outside itself. React components are written exactly like this.

DO

```
function Price({ amount }) {
  return <p>{amount * 1.1}</p>;
}
```

DON'T

```
let total = 0;
function Price({ amount }) {
  total += amount; // outside change!
```

LC

TOPIC 1

let & const

Declare variables the safe, modern way.

let and const replace the old var. const makes a name that cannot be reassigned, let allows reassignment, and both live only inside the block { } where they are created.

KEY POINTS

- const is the default choice; use let only when a value must change
- Never use var: it is function-scoped and hoists confusingly
- Both are block-scoped: they live inside the nearest { }
- const stops reassignment, not changes inside objects or arrays
- React state follows the same idea: you replace, never reassign

```
// const cannot be reassigned
const course = 'React';
course = 'Vue'; // TypeError!

// let can change
let count = 0;
count = count + 1;

// block scope
if (true) {
  const secret = 42;
}
console.log(secret); // ReferenceError

// const object: inside can change
const user = { age: 19 };
user.age = 20; // allowed
```

> *Start with const. Switch to let only when you truly need to reassign.*

=>

TOPIC 2

Arrow Functions

A shorter way to write functions.

An arrow function is a compact function syntax that uses =>. It is everywhere in React: event handlers, array methods, and even function components themselves.

KEY POINTS

- => replaces the function keyword; one expression returns automatically
- Use { } with return when the body has several statements
- Wrap a returned object in ( ) so it is not read as a block
- Arrow functions keep the this of their surroundings
- Perfect for inline handlers and callbacks like map and filter

```
// Regular function
function add(a, b) {
  return a + b;
}

// Arrow function
const add = (a, b) => a + b;

// One parameter: no brackets needed
const double = n => n * 2;

// Returning an object: wrap in ( )
const makeUser = name => ({ name });

// In React
<button onClick={() => setOpen(true)}>
  Open
</button>
```

> *Expression body = implicit return. Curly body = write return yourself.*

TOPIC 3

${}

Template Literals

Strings that can hold variables and span lines.

Template literals use backticks instead of quotes. You can drop any JavaScript expression inside ${ } and write multi-line text naturally. React uses them for class names, URLs and messages.

KEY POINTS

- Written with backticks ` ` instead of quotes
- Put any expression inside ${ } and its result becomes text
- Multi-line strings need no \n characters
- Great for dynamic URLs such as `/posts/${id}`
- In JSX, only the plain { } is used for values; ${ } is for strings

```
const name = 'Sara';
const items = 3;

// Old way
'Hi ' + name + ', you have ' + items + ' items'

// Template literal
`Hi ${name}, you have ${items} items`

// Any expression works
`Total: ${items * 10}`

// Multi-line
const msg = `Line one
Line two`;

// Dynamic className in React
<div className={`card ${on ? 'active' : ''}`}>
```

> *Backtick + ${ } = a string with holes you fill in.*

TOPIC 4

Destructuring

Unpack values from objects and arrays in one line.

Destructuring pulls values out of an object or array into separate variables. React uses it every day: for props, for the result of useState, and for API data.

KEY POINTS

- Objects match by NAME with { }; arrays match by POSITION with [ ]
- Rename with name: newName
- Give a default with = for missing values
- Skip array items using empty commas
- Props and useState both rely on it constantly

```
// Object destructuring (by name)
const user = { name: 'Sara', age: 20 };
const { name, age } = user;

// Rename + default value
const { name: fullName, city = 'Paris' } = user;

// Array destructuring (by position)
const [first, second] = ['a', 'b'];

// Skip an item
const [, onlySecond] = [1, 2];

// In React
function Card({ title, price }) { ... }
const [count, setCount] = useState(0);
```

> *Curly braces match by name. Square brackets match by position.*

TOPIC 5

Spread & Rest

Three dots, two meanings: expand or collect.

The … operator spreads items out when you copy or combine, and gathers items together (rest) when you receive them. It is the main tool for updating React state without mutating it.

KEY POINTS

- Spread unpacks; rest packs the leftovers together
- Copies are shallow: nested objects are still shared
- Later keys win: { …user, age: 21 } overrides age
- The classic state update: setItems([…items, newItem])
- A rest item must always come last

```
// SPREAD: copy and extend an array
const a = [1, 2];
const b = [...a, 3]; // [1, 2, 3]

// SPREAD: copy and change an object
const user = { name: 'Sara', age: 20 };
const older = { ...user, age: 21 };

// REST: collect the remaining items
function sum(...nums) {
  return nums.reduce((t, n) => t + n, 0);
}

// REST in destructuring
const { name, ...others } = user;

// In React state
setItems([...items, newItem]);
```

> *Spread opens the box. Rest fills a new box.*

TOPIC 6

MAP

Array Methods

Transform lists without writing loops.

map, filter and reduce return new arrays or values and never change the original. They are how React turns plain data into lists of UI.

KEY POINTS

- map transforms every item and returns a same-length array
- filter keeps only the items that pass a test
- reduce folds the list into one value, like a total
- find returns the first match; some / every return true or false
- None of them mutate the original array

```
const nums = [1, 2, 3, 4];

nums.map(n => n * 2);    // [2, 4, 6, 8]
nums.filter(n => n % 2 === 0); // [2, 4]
nums.reduce((t, n) => t + n, 0); // 10
nums.find(n => n > 2);   // 3
nums.some(n => n > 3);   // true
nums.every(n => n > 0);  // true

// In React: array to UI
{users.map(u => (
  <li key={u.id}>{u.name}</li>
))}
```

> *Rendering a list in React is just map() that returns JSX.*

TOPIC 7

?.

Optional Chaining & ??

Read deep values safely without crashing.

Optional chaining (?.) stops and returns undefined when something is missing, instead of throwing an error. The nullish operator (??) supplies a fallback only for null or undefined.

KEY POINTS

- ?. returns undefined the moment a link is null or undefined
- Works on properties, methods ?.() and indexes ?.[ ]
- ?? falls back only for null or undefined
- || falls back for any falsy value: 0, empty string, false
- Essential when API data has not arrived yet

```
const user = { profile: null };

// Without ?.  crashes
user.profile.name; // TypeError

// With ?.  safe
user.profile?.name; // undefined

// Optional method and index
user.getAge?.();
list?.[0];

// ?? versus ||
const qty = 0;
qty || 10; // 10 (0 is falsy)
qty ?? 10; // 0  (only null / undefined)

<p>{user?.address?.city ?? 'Unknown'}</p>
```

> *Use ?? for defaults when 0 or an empty string is a real value.*

IM

TOPIC 8

Modules

Split code into files and share it with import / export.

Every file is a module. You export what other files may use and import what you need. In React, each component normally lives in its own module.

KEY POINTS

- Default export: one per file, imported with any name
- Named exports: many per file, imported in { } with the exact name
- Your own files need ./ or ../ ; packages like react do not
- Keep one component per file, named like the component
- Files that contain JSX use the .jsx extension

```jsx
// Button.jsx
export default function Button() {
  return <button>Click</button>;
}
export const SIZE = 'md'; // named

// App.jsx
import Button, { SIZE } from './Button';
import { useState } from 'react';
import './App.css';

// Rename on import
import { SIZE as size } from './Button';
```

> *Default = the main thing in the file. Named = the extras.*

AW

TOPIC 9

Promises & async / await

Wait for slow work without freezing the screen.

Some tasks take time, like fetching data. A Promise stands for a future result, and async / await lets you write that waiting code so it reads top to bottom like normal code.

KEY POINTS

- A Promise is pending, then fulfilled or rejected
- await pauses inside an async function until the promise settles
- Wrap awaits in try / catch to handle errors
- fetch needs two awaits: one for the response, one for res.json()
- In useEffect, define the async function inside and call it

```
// Promise style
fetch('/api/users')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// async / await style
async function getUsers() {
  try {
    const res = await fetch('/api/users');
    const data = await res.json();
    return data;
  } catch (err) {
    console.error(err);
  }
}
```

> *await means: wait here, but do not block the rest of the page.*

TOPIC 10

V

Vite & npm Tooling

Set up a React project in under a minute.

Node runs JavaScript outside the browser and npm installs packages. Vite is the fast tool that creates, runs and builds your React app.

KEY POINTS

- Node and npm install together from nodejs.org
- package.json lists your dependencies and scripts
- node_modules holds installed packages; never edit or commit it
- npm run dev starts a hot-reloading dev server
- npm run build creates optimised files in the dist folder

```
# 1. Create the project
npm create vite@latest my-app -- --template react

# 2. Install and start
cd my-app
npm install
npm run dev   # http://localhost:5173

# 3. Build for production
npm run build

# Key files
index.html    the single page
src/main.jsx  starts React
src/App.jsx   your first component
package.json  scripts + dependencies
```

> *Daily loop: edit, save, and watch the browser update instantly.*

QUICK REFERENCE

Foundations Cheat Sheet

CONCEPT	PURPOSE	USE IT WHEN…
let / const	Declare block-scoped variables	Every variable (const by default)
Arrow functions	Short function syntax	Handlers, callbacks, components
Template literals	Strings with embedded values	Dynamic classNames, URLs, messages
Destructuring	Unpack objects and arrays	Props, useState, API data
Spread & rest	Copy / merge, or collect leftovers	Updating state immutably
map / filter / reduce	Transform lists	Rendering lists, filtering, totals
?. and ??	Safe access and fallbacks	API data may be missing
import / export	Share code between files	Every component file
async / await	Wait for promises cleanly	Fetching data from an API
Vite + npm	Create, run and build the app	Starting any React project



Thank You

Questions about any topic? Let's discuss.

let / const · Arrow functions · Destructuring · Spread · Array methods · async / await · Vite

NEXT: PART 2 — CORE REACT: JSX, COMPONENTS & PROPS

REACT · LECTURE SERIES · PART 2

Core React

A complete, in-depth, simple-English guide to the building blocks of every React screen — JSX, components, props, children, lists, conditions & events.

JSX • Components • Props • Events



LECTURE ROADMAP

What We'll Cover Today

JSX
JSX Basics
HTML-like syntax inside JavaScript

{ }
Expressions in JSX
Put JavaScript values inside markup

AT
Attributes & className
Pass data and styles to elements

<>
Fragments
Group elements without extra HTML

FC
Function Components
A function that returns JSX

PR
Props
Pass data into a component

CH
children & Composition
Nest components inside each other

LK
Lists & Keys
Render many items from an array

?:
Conditional Rendering
Show different UI depending on data

EV
Events
Respond to clicks, typing and submits

THE BIG IDEA

What Exactly Is a Component?

A component is a small, reusable piece of the screen, written as a JavaScript function. A whole app is just a tree of components: a Navbar, a Card, a Button, all combined together.

> IN ONE LINE:
>
> ***"Build the UI once as a component, then reuse it everywhere with different data."***

Why components matter:

- Reuse: write a Card once, use it fifty times
- Isolation: fix one piece without breaking others
- Readability: each file does one clear job
- Teamwork: people can build different pieces

BEFORE — Repeated HTML

```
<div class="card"><h2>Pen</h2></div>
<div class="card"><h2>Book</h2></div>
<div class="card"><h2>Bag</h2></div>

// copy, paste, edit every one
// change the design = change all 3
```

AFTER — One Component

```
function Card({ title }) {
  return <h2>{title}</h2>;
}

<Card title="Pen" />
<Card title="Book" />
```

BEFORE YOU START

The Two Golden Rules of Components

1 Names start with a capital letter

React treats lowercase tags as plain HTML elements. A capital first letter is how it knows that Card is YOUR component and not an HTML tag.

DO

```
function Card() { return <p>Hi</p>; }
<Card />
```

DON'T

```
function card() { return <p>Hi</p>; }
<card />  // treated as HTML
```

2 Return one parent element

A component can only return a single root. Wrap siblings in a div or, better, in an empty Fragment so no extra HTML is added.

DO

```
return (
  <>
    <h1>Title</h1> <p>Text</p>
  </>
);
```

DON'T

```
return (
  <h1>Title</h1>
  <p>Text</p>   // two roots!
);
```

JSX

TOPIC 1

JSX Basics

HTML-like syntax inside JavaScript.

JSX looks like HTML but it is really JavaScript. Tools like Vite convert every JSX tag into a plain JavaScript call, so you can describe your UI right inside your code.

KEY POINTS

- JSX is not HTML; it is converted into JavaScript function calls
- Every tag must be closed, even <img /> and
- Return one parent element, and wrap multi-line JSX in ( )
- Vite (via Babel) does the conversion; browsers cannot read JSX
- Using JSX is optional, but almost every React project uses it

```
// JSX (what you write)
const el = <h1 className="title">Hello</h1>;

// What it becomes (simplified)
const el = React.createElement(
  'h1',
  { className: 'title' },
  'Hello'
);

// A component returns JSX
function App() {
  return (
    <main>
      <h1>Hello, React</h1>
    </main>
  );
}
```

> *JSX is just a nicer way to write React.createElement().*

TOPIC 2

Expressions in JSX

Put JavaScript values inside your markup.

Curly braces { } open a window from JSX back into JavaScript. Anything that produces a value can go inside: variables, math, function calls and ternaries.

KEY POINTS

- { } accepts expressions: anything that produces a value
- Statements like if and for are not allowed inside { }
- null, undefined, true and false render nothing
- Objects cannot be rendered directly; show their fields instead
- Strings and numbers appear as normal text

```
const name = 'Sara';
const price = 40;

return (
  <div>
    <h2>Hello, {name}</h2>
    <p>Total: {price * 3}</p>
    <p>{name.toUpperCase()}</p>
    <p>{price > 30 ? 'Pricey' : 'Cheap'}</p>
    <p>{`Items: ${3}`}</p>
  </div>
);

// NOT allowed: statements
// {if (x) { ... }}
// {for (...) { ... }}
```

> *If it can go on the right side of an = sign, it can go inside { }.*

TOPIC 3

AT

Attributes & className

Pass data and styles to elements.

JSX attributes are written in camelCase because they are really JavaScript properties. A few names differ from HTML, and style takes an object instead of a string.

KEY POINTS

- class becomes className, and for becomes htmlFor
- Multi-word attributes use camelCase: onClick, tabIndex
- style takes an object, so you see double braces {{ }}
- Use quotes for fixed text and { } for live values
- Self-closing tags need the slash: <input />

```
// HTML        becomes   JSX
// class                   className
// for                     htmlFor
// onclick                 onClick
// tabindex                tabIndex

<label htmlFor="email">Email</label>
<input id="email" type="email" />

<div style={{ color: 'teal', fontSize: 18 }}>
  Styled text
</div>

// Dynamic value
<img src={user.avatar} alt={user.name} />
```

> *Quotes for fixed text. Curly braces for live values.*

TOPIC 4

Fragments

Group elements without adding extra HTML.

A component must return a single parent. A Fragment is an invisible wrapper that satisfies that rule without adding an extra div to the page.

KEY POINTS

- <> </> is the short Fragment syntax
- Use <React.Fragment key={id}> when a key is needed
- No extra DOM element, so flex, grid and tables stay clean
- Handy when a component returns several table cells or rows
- Fewer wrapper divs means simpler CSS

```
// Problem: two siblings
// return <h1>Title</h1><p>Text</p>; // error

// Fix 1: extra div (adds markup)
return (
  <div>
    <h1>Title</h1>
    <p>Text</p>
  </div>
);

// Fix 2: Fragment (no extra markup)
return (
  <>
    <h1>Title</h1>
    <p>Text</p>
  </>
);
```

> *Fragments: all the grouping, none of the extra HTML.*

FC

TOPIC 5

Function Components

A component is a function that returns JSX.

A component is a JavaScript function whose name starts with a capital letter and that returns JSX. You use it like an HTML tag, and React calls it for you.

KEY POINTS

- The name must start with a capital letter
- Return JSX, or null to show nothing
- Use it like a tag: <Welcome /> or with children inside
- Keep one component per file, exported and imported
- React calls your function again on every render

```
// Define
function Welcome() {
  return <h1>Welcome to React</h1>;
}

// Arrow function version
const Footer = () => <p>Made with React</p>;

// Use them like tags
function App() {
  return (
    <>
      <Welcome />
      <Footer />
    </>
  );
}
```

> *A screen is a tree of small functions calling each other.*

TOPIC 6

PR

Props

Pass data into a component like function arguments.

Props (properties) are the inputs of a component. The parent passes them as attributes, and the child receives them as one object that you usually destructure.

KEY POINTS

- Props flow one way: from parent to child
- Props are read-only; a component never changes its own props
- Strings go in quotes, everything else in { }
- A prop with no value (inStock) means true
- Set default values with = while destructuring

```
// Parent passes props
<Card title="Pen" price={20} inStock />

// Child receives them (destructured)
function Card({ title, price, inStock = true }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <p>${price}</p>
      {inStock && <span>Available</span>}
    </div>
  );
}

// Same without destructuring
function Card(props) {
  return <h2>{props.title}</h2>;
}
```

> *Props flow DOWN. Events flow UP. Props are read-only.*

TOPIC 7

CH

children & Composition

Nest components inside each other.

Anything you put between a component's opening and closing tags arrives as the special children prop. This is how wrappers like cards, modals and layouts are built.

KEY POINTS

- children is a built-in prop holding whatever is nested inside
- It can be text, elements or other components
- Composition: build big UI by combining small components
- Wrappers such as Layout and Modal rely on children
- You can also pass JSX in named props: icon={<Star />}

```
function Card({ title, children }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      {children}
    </div>
  );
}

// Usage: put anything inside
<Card title="Profile">
  <img src="me.png" alt="Me" />
  <p>A short bio</p>
</Card>

// Composition
<Page><Navbar /><Card /></Page>
```

> *Think of children as the empty slot in a picture frame.*

LK

TOPIC 8

Lists & Keys

Render many items from an array.

To show a list, map over an array and return an element for each item. Each item needs a unique key so React can track which one changed, moved or was removed.

KEY POINTS

- map turns data into JSX elements
- The key goes on the outermost element inside map
- Keys must be unique among siblings and stable (use an id)
- Index keys are only safe for lists that never change order
- Filter first, then map, to show a subset

```
const users = [
  { id: 1, name: 'Sara' },
  { id: 2, name: 'Ali' },
  { id: 3, name: 'Noor' },
];

function UserList() {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}

// Avoid index as key if list can reorder
```

> *A key is a name tag: it tells React which item is which.*

TOPIC 9

?:

Conditional Rendering

Show different UI depending on the data.

React has no special if tag. You use normal JavaScript: early returns, ternaries or the logical && operator to decide what a component shows.

KEY POINTS

- Ternary for either / or choices
- && for show-it-or-nothing
- Early return for guard cases: loading, empty, error
- Careful: {0 && <X />} renders the number 0; compare with > 0
- null, false and undefined render nothing

```
function Status({ loggedIn, count }) {
  // 1. Early return
  if (!count) return null;

  return (
    <div>
      {/* 2. Ternary: this or that */}
      {loggedIn ? <Dashboard /> : <Login />}

      {/* 3. && : this or nothing */}
      {count > 0 && <Badge n={count} />}
    </div>
  );
}
```

> *Ternary = this or that. && = this or nothing.*

TOPIC 10

EV

Events

Respond to clicks, typing and form submits.

React events look like HTML events but use camelCase and receive a function, not a string. React wraps the browser event in a synthetic event that behaves the same in every browser.

KEY POINTS

- camelCase names: onClick, onChange, onSubmit
- Pass the function itself: onClick={fn}, not onClick={fn()}
- Need arguments? Use an arrow: onClick={() => remove(id)}
- e.preventDefault() stops the page from reloading on submit
- e.target.value reads what the user typed

```
function Form() {
  const handleChange = (e) => {
    console.log(e.target.value);
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Sent');
  };

  return (
    <form onSubmit={handleSubmit}>
      <input onChange={handleChange} />
      <button type="submit">Send</button>
    </form>
  );
}
```

> *Pass the function itself. Do not call it while rendering.*

QUICK REFERENCE

Core React Cheat Sheet

CONCEPT	PURPOSE	USE IT WHEN…
JSX	HTML-like syntax in JavaScript	Describing UI inside components
{ } expressions	Insert JS values into JSX	Variables, math, ternaries
Attributes	Pass settings to elements	className, htmlFor, style, onClick
Fragment <></>	Group without extra markup	Returning several siblings
Function component	Reusable piece of UI	Every part of your screen
Props	Inputs from parent to child	Passing data and callbacks down
children	A slot for nested content	Cards, modals, layouts
map + key	Render arrays as lists	Users, products, posts
Conditional rendering	Choose what to show	Login state, empty and loading UI
Events	Respond to user actions	Clicks, typing, form submit



Thank You

Questions about any topic? Let's discuss.

JSX · Components · Props · children · Lists & keys · Conditions · Events

NEXT: PART 3 — STYLING & UI

REACT · LECTURE SERIES · PART 3

Styling & UI

A complete, in-depth, simple-English guide to making React apps look great — CSS, CSS Modules, Tailwind, layout, responsive design, dark mode, icons, animation & accessibility.

CSS Modules • Tailwind • Responsive • Accessibility



LECTURE ROADMAP

What We'll Cover Today

CSS
Plain CSS in React
Import stylesheets into components

MOD
CSS Modules
Scoped class names, no clashes

TW
Tailwind CSS
Utility classes right inside JSX

ST
Inline Styles
Style objects for dynamic values

FG
Flexbox & Grid
Modern layout with two tools

RS
Responsive Design
Mobile-first for every screen

DM
Dark Mode
Themes with CSS variables

IC
Icons & UI Libraries
lucide, shadcn/ui and MUI

AN
Animation
Smooth motion with Motion

A11Y
Accessibility
Interfaces usable by everyone

THE BIG IDEA

How Does Styling Work in React?

React does not care how you style. You choose a method: plain CSS files, CSS Modules, Tailwind, inline styles or a UI library. The real challenge is keeping styles from clashing as the app grows.

> IN ONE LINE:
>
> ***"Style each component so it looks right on its own and never breaks its neighbours."***

What good styling gives you:

- Consistency: same colours and spacing everywhere
- Safety: one component cannot break another
- Responsiveness: works on phone, tablet, desktop
- Accessibility: usable with keyboard and screen reader

BEFORE — Global CSS Clash

```
/* a.css */  .title { color: red; }
/* b.css */  .title { color: blue; }

// both files load for the whole app
// last one wins EVERYWHERE
```

AFTER — CSS Module

```
// Card.module.css  =>  .title { ... }
import styles from './Card.module.css';

<h2 className={styles.title}>Hi</h2>
// real class: Card_title__x7a2k
// unique, so no clashes
```

BEFORE YOU START

The Two Golden Rules of Styling

1 Use classes first, inline last

Classes are reusable and support hover, focus and media queries. Keep inline style for values that truly change at runtime, like a progress width.

DO

```
<div className="card">
<div style={{ width: `${pct}%` }}>
```

DON'T

```
<div style={{ padding: 16, color: "teal",
  borderRadius: 12 }}>  // repeat everywhere
```

2 Design mobile first

Write the small-screen layout as the default, then add rules for larger screens with min-width queries. It is simpler and loads leaner CSS on phones.

DO

```
.grid { display: grid; }
@media (min-width: 768px) {
  .grid { grid-template-columns: 1fr 1fr; }
}
```

DON'T

```
.grid { grid-template-columns: 1fr 1fr 1fr; }
@media (max-width: 767px) {
  .grid { grid-template-columns: 1fr; }
}
```

CSS

TOPIC 1

Plain CSS in React

Import a stylesheet straight into a component.

With Vite you simply import a .css file and its rules are added to the page. It is the simplest way to style, but remember: plain CSS is global, even when imported in one component.

KEY POINTS

- Vite injects every imported CSS file into the page
- Even if imported in one file, the CSS applies to the whole app
- Use className, not class, in JSX
- Choose unique class names (card__title) to avoid clashes
- Keep index.css for resets, fonts and CSS variables

```
// main.jsx: global styles, once
import './index.css';

// Card.jsx: this component's file
import './Card.css';

/* Card.css */
.card {
  padding: 16px;
  border-radius: 12px;
}
.card:hover { box-shadow: 0 4px 12px #0003; }

// Card.jsx
<div className="card">Hello</div>

// Dynamic class
<div className={`card ${on ? 'active' : ''}`}>
```

> *Plain CSS is global. Name classes carefully.*

MOD

TOPIC 2

CSS Modules

Class names that belong to one component only.

A CSS Module is a normal CSS file whose class names are made unique automatically. You import it as an object and use its keys, so two components can both have a .title without clashing.

KEY POINTS

- The file name must end with .module.css
- Vite renames each class to a unique name automatically
- Import as an object and write styles.className
- Use camelCase names (cardTitle) so dot access works
- Works out of the box in Vite, no setup needed

```
/* Card.module.css */
.card { padding: 16px; }
.title { color: teal; }
.active { border: 2px solid teal; }

// Card.jsx
import styles from './Card.module.css';

function Card({ active }) {
  return (
    <div className={styles.card}>
      <h2 className={styles.title}>Hi</h2>
    </div>
  );
}

// Combine two classes
className={`${styles.card} ${styles.active}`}
```

> *Same class name, different files, zero conflicts.*

TOPIC 3

TW

Tailwind CSS

Style with small utility classes inside JSX.

Tailwind gives you tiny single-purpose classes like p-4, flex and text-lg. You build a design by combining them right in your markup, with no separate CSS file for most things.

KEY POINTS

- Small classes combine: p-4, text-lg, flex, rounded
- Prefixes add states and screens: hover:, focus:, md:, dark:
- Tailwind v4 setup is the Vite plugin plus one @import line
- Older versions used tailwind.config.js and @tailwind directives
- Repeated class groups belong in a component, not copy-pasted

```
# Install (Tailwind v4 + Vite)
npm install tailwindcss @tailwindcss/vite

// vite.config.js
plugins: [react(), tailwindcss()],

/* index.css */
@import "tailwindcss";

// Use utility classes
<button className="rounded bg-teal-600 px-4 text-white">
  Save
</button>

// State and screen prefixes
<p className="text-lg hover:text-teal-600 md:text-2xl">
```

> *Utility first: compose small classes instead of writing new CSS.*

ST

TOPIC 4

Inline Styles

A style object for values that change.

The style prop takes a JavaScript object, not a string. Keys are camelCase and plain numbers become pixels. Inline styles are perfect for values calculated at runtime.

KEY POINTS

- Double braces: one for JSX, one for the object
- Keys are camelCase: backgroundColor, not background-color
- Plain numbers become px; use strings for other units
- No :hover, no media queries, no pseudo-elements
- Use sparingly; prefer classes for anything reusable

```
// Object, camelCase, numbers = px
<p style={{ fontSize: 18, marginTop: 8 }}>Hi</p>

// Best use: values that change
<div
  style={{ width: `${progress}%` }}
  className="bar"
/>

// Pull the object out
const box = { padding: 16, color: 'teal' };
<div style={box}>Boxed</div>

// Dynamic colour from data
<span style={{ color: ok ? "green" : "red" }}>
  Status</span>

// Cannot do: :hover, @media, ::before
```

> *Inline style = values that change. Classes = everything else.*

FG

TOPIC 5

Flexbox & Grid

Two tools that handle almost every layout.

Flexbox arranges items along one direction, a row or a column. Grid arranges items in rows and columns at the same time. Together they replace most old float and position tricks.

KEY POINTS

- Flexbox: navbars, button rows, centering, one-dimensional layout
- Grid: card galleries, page layouts, two-dimensional layout
- gap adds space between items without margin hacks
- repeat(3, 1fr) makes three equal columns
- Both are easy to switch on inside any React component

```css
/* Flexbox: one direction */
.row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 16px;
}

/* Grid: rows and columns */
.cards {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}

/* Center anything */
.center { display: grid; place-items: center; }
```

> *One direction? Flexbox. Rows and columns? Grid.*

TOPIC 6

RS

Responsive Design

One design that adapts to every screen.

Responsive design means the same page rearranges itself for phones, tablets and desktops. Start with the small-screen layout, then add media queries as the screen gets wider.

KEY POINTS

- Mobile first: base styles for phones, min-width queries upward
- Common breakpoints: 640, 768, 1024 and 1280 pixels
- Use relative units: rem, %, vh, and clamp() for fluid text
- Make images flexible with max-width: 100%
- Always test on a real phone or DevTools device mode

```
/* Mobile first: default = phone */
.grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}
@media (min-width: 640px) {
  .grid { grid-template-columns: repeat(2, 1fr); }
}
@media (min-width: 1024px) {
  .grid { grid-template-columns: repeat(3, 1fr); }
}

// Same idea with Tailwind
<div className="grid grid-cols-1 gap-4
  sm:grid-cols-2 lg:grid-cols-3">
```

> *Start small, then add rules as the screen grows.*

TOPIC 7

DM

Dark Mode

Switch whole themes by changing CSS variables.

Define your colours as CSS variables once. A dark theme is then just a second set of values, switched on by an attribute on the page. React only needs to flip that attribute.

KEY POINTS

- Variables like --bg are defined once and reused everywhere
- A data-theme attribute on <html> picks the active set
- Save the choice in localStorage so it survives refresh
- prefers-color-scheme reads the visitor's system setting
- Tailwind: the dark: prefix styles dark mode

```
:root {
  --bg: #ffffff;
  --text: #111111;
}
[data-theme='dark'] {
  --bg: #10131f;
  --text: #f2f2f2;
}
body { background: var(--bg); color: var(--text); }

// React: toggle the attribute
const [theme, setTheme] = useState('light');

useEffect(() => {
  document.documentElement.dataset.theme = theme;
}, [theme]);

prefers-color-scheme: dark  // OS setting
```

> *Change the variables, and the whole app changes.*

IC

TOPIC 8

Icons & UI Libraries

Ready-made icons and components to move faster.

Icon packs give you SVG icons as React components. UI libraries give you tested, accessible building blocks like buttons, modals and tables so you do not start from zero.

KEY POINTS

- Icons are components; size and color are props
- react-icons bundles many icon sets in one package
- shadcn/ui copies component source into your project, so you own it
- MUI, Chakra and Mantine are installed as dependencies
- Pick ONE library and stay consistent

```
# Install an icon pack
npm install lucide-react

import { Heart, Search } from 'lucide-react';

<Heart size={20} color="crimson" />
<Search className="h-5 w-5" />

// Component libraries
// shadcn/ui : copy-in parts + Tailwind
// MUI       : Material Design components
// Chakra, Mantine, Ant Design : more options

import Button from '@mui/material/Button';
<Button variant="contained">Save</Button>
```

> *Do not rebuild a modal from scratch unless you must.*

AN

TOPIC 9

Animation

Smooth motion with the Motion library.

Motion (formerly Framer Motion) adds animation to React with simple props. You describe where an element starts and where it ends, and the library handles the smooth movement between.

KEY POINTS

- motion.div is a normal div with animation powers
- initial and animate describe the start and end state
- whileHover and whileTap animate gestures
- Exit animations need the AnimatePresence wrapper
- Animate opacity and transform; respect reduced-motion settings

```
// npm install motion
import { motion } from 'motion/react';

<motion.div
  initial={{ opacity: 0, y: 20 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.5 }}
>
  Hello
</motion.div>

<motion.button
  whileHover={{ scale: 1.05 }}
  whileTap={{ scale: 0.95 }}
>Tap me</motion.button>
```

> *Describe start and end. Motion handles the in-between.*

A11Y

TOPIC 10

Accessibility

Interfaces everyone can use, including by keyboard.

Accessibility (a11y) means people using screen readers, keyboards or low vision can use your app. Good semantic HTML solves most of it before you write any extra code.

KEY POINTS

- Use the right tag: button, nav, main, h1 to h6
- Every image needs alt text; decorative ones get alt=""
- Connect labels to inputs with htmlFor and id
- Keep a visible focus outline and test with the Tab key
- Text contrast should be at least 4.5 to 1

```
// Semantic tags, not div soup
<nav>  <main>  <header>  <footer>

// Real button = keyboard + screen reader
<button onClick={open}>Menu</button>
// not: <div onClick={open}>Menu</div>

<img src="cat.png" alt="Orange cat on a sofa" />
<img src="line.svg" alt="" />  // decorative

<label htmlFor="mail">Email</label>
<input id="mail" type="email" />

<button aria-label="Close dialog">X</button>
```

> *If you cannot use it with only a keyboard, fix it.*

QUICK REFERENCE

Styling & UI Cheat Sheet

CONCEPT	PURPOSE	USE IT WHEN…
Plain CSS	Global stylesheets	Resets, variables, small apps
CSS Modules	Scoped class names	Component styles without clashes
Tailwind CSS	Utility classes in JSX	Fast, consistent design
Inline style	Style object on an element	Values that change at runtime
Flexbox & Grid	Modern page layout	Navbars, rows, card grids
Media queries	Change styles by screen size	Responsive, mobile-first layouts
CSS variables	Reusable design tokens	Themes and dark mode
Icons / UI kits	Ready-made visuals	Building interfaces quickly
Motion	Animate elements	Entrances, hover, transitions
Accessibility	Usable by everyone	Every single component



Thank You

Questions about any topic? Let's discuss.

CSS · CSS Modules · Tailwind · Flexbox & Grid · Responsive · Dark mode · Motion · a11y

NEXT: PART 4 — STATE & FORMS

REACT · LECTURE SERIES · PART 4

State & Forms

A complete, in-depth, simple-English guide to making React interactive — state, updater functions, objects and arrays, controlled inputs, form handling, lifting state & validation.

useState in depth • Forms • Validation



LECTURE ROADMAP

What We'll Cover Today

ST
State Basics
Remember data between renders

UP
Updater Functions
Update from the latest value

OB
Objects in State
Copy with spread, never mutate

AR
Arrays in State
Add, remove and update safely

IN
Controlled Inputs
State drives the input value

CB
Checkbox, Radio & Select
The other form controls

SB
Submitting Forms
One handler for many fields

LU
Lifting State Up
Share state through a parent

DR
Derived State
Calculate it, do not store it

VA
Validation & Hook Form
Check input before sending

THE BIG IDEA

What Exactly Is State?

State is data that a component remembers between renders and that can change over time, like a counter, the text in an input, or whether a menu is open. When state changes, React redraws the component.

> IN ONE LINE:
>
> ***"State is a component's memory. Change it, and React redraws the screen."***

Why state exists:

- The UI always stays in sync with the data
- React knows exactly WHEN to re-render
- Every component instance has its own memory
- It makes interactive screens possible

BEFORE — Plain Variable

```
let count = 0;
function add() {
  count = count + 1;
}

// count changes, screen still shows 0
```

AFTER — useState

```
const [count, setCount] = useState(0);
function add() {
  setCount(count + 1);
}

// React re-renders, screen shows 1
```

BEFORE YOU START

The Two Golden Rules of State

1 Change state only with its setter

React cannot notice a direct assignment, so the screen would not update. Always call the setter function, and give it a new value or a new copy.

DO

```
setCount(count + 1);
setUser({ ...user, age: 21 });
```

DON'T

```
count = count + 1;
user.age = 21;   // no re-render
```

2 Store the minimum, derive the rest

If a value can be calculated from existing state or props, calculate it while rendering. Duplicate state drifts out of sync.

DO

```
const [items, setItems] = useState([]);
const count = items.length;
```

DON'T

```
const [items, setItems] = useState([]);
const [count, setCount] = useState(0);
// two sources of truth
```

ST

TOPIC 1

State Basics

Give a component memory that triggers redraws.

useState returns the current value and a setter function. Calling the setter stores the new value and tells React to render the component again with it.

KEY POINTS

- State is data a component remembers between renders
- The setter saves the new value and triggers a re-render
- Ordinary variables reset on every render and update nothing
- Each component instance has its own, private state
- The initial value is used only on the very first render

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Add
      </button>
    </div>
  );
}

// Plain variable: UI never updates
let n = 0;  // n++ does NOT re-render
```

> *Change state: React redraws. Change a variable: nothing happens.*

UP

TOPIC 2

Updater Functions

Update state using its latest value.

Inside one render, state is a fixed snapshot. If the new value depends on the old one, pass the setter a function that receives the latest value instead of reading the variable.

KEY POINTS

- State is a snapshot: every render sees one fixed value
- setX(value) uses that snapshot, so repeated calls overwrite
- setX(prev => ...) receives the newest pending value
- Use the updater form whenever new state depends on old state
- React batches several updates into a single re-render

```
const [count, setCount] = useState(0);

// Snapshot problem: count is 0 in all 3
function addThree() {
  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1); // result: 1
}

// Updater function: latest value
function addThreeRight() {
  setCount(prev => prev + 1);
  setCount(prev => prev + 1);
  setCount(prev => prev + 1); // result: 3
}

// Lazy initial value (runs once)
useState(() => loadFromStorage());
```

> *New value depends on the old one? Pass a function.*

TOPIC 3

OB

Objects in State

Replace the object, never edit it.

React decides whether to re-render by comparing references. If you edit an object in place, the reference stays the same and React sees nothing new. Always create a fresh copy with spread.

KEY POINTS

- Never edit a state object directly
- React compares by reference, so you need a new object
- Spread copies the top level; the later key wins
- Nested objects need a copy at every level you change
- Prefer flat state when you can: setUser(p => ({ ...p, age: 21 }))

```
const [user, setUser] = useState({
  name: 'Sara',
  age: 20,
});

// Wrong: mutates the same object
user.age = 21;   // no re-render!
setUser(user);

// Right: new object with spread
setUser({ ...user, age: 21 });

// Nested object: copy each level
setUser({
  ...user,
  address: { ...user.address, city: 'Paris' },
});
```

> *Spread to copy, override what changed, set the copy.*

AR

TOPIC 4

Arrays in State

Add, remove and update without mutating.

The same rule applies to lists. Methods that change an array in place, like push and splice, are off limits. Use methods that return a brand new array instead.

KEY POINTS

- Add: spread into a new array
- Remove: filter out the item
- Update: map and replace the matching item
- Avoid push, pop, splice, reverse and sort on state directly
- Need to sort? Copy first: [...items].sort()

```
const [items, setItems] = useState([]);

// Add
setItems([...items, newItem]);

// Remove
setItems(items.filter(i => i.id !== id));

// Update one item
setItems(items.map(i =>
  i.id === id ? { ...i, done: true } : i
));

// Insert at the start
setItems([newItem, ...items]);

// Avoid: push, pop, splice, sort
// items.push(newItem)  // mutates!
```

> *Add = spread. Remove = filter. Update = map.*

IN

TOPIC 5

Controlled Inputs

Let React own what the input shows.

A controlled input gets its value from state and reports every keystroke back through onChange. State becomes the single source of truth, so you can validate or format as the user types.

KEY POINTS

- value comes from state; onChange writes the change back
- State is the single source of truth for the field
- You can validate, limit or format input instantly
- A value without onChange makes the input read-only
- Start with an empty string, not undefined, to stay controlled

```
const [name, setName] = useState('');

return (
  <>
    <input
      value={name}
      onChange={e => setName(e.target.value)}
    />
    <p>Hello, {name}</p>
  </>
);

// Typing -> onChange -> setName
// -> re-render -> input shows new value
```

> *The input shows state. Typing updates state. Loop complete.*

CB

TOPIC 6

Checkbox, Radio & Select

Other controls follow the same pattern.

Every form control can be controlled. Text inputs, selects and textareas use value, while checkboxes and radio buttons use checked. The handler reads the matching property from the event.

KEY POINTS

- Text, select and textarea use value
- Checkbox and radio use checked
- Checkbox reads e.target.checked; others read e.target.value
- A radio group shares one state; each radio compares its value
- On a select, value goes on <select>, not on <option>

```
const [agree, setAgree] = useState(false);
const [plan, setPlan] = useState('free');
const [city, setCity] = useState('paris');

// Checkbox uses checked
<input type="checkbox" checked={agree}
  onChange={e => setAgree(e.target.checked)} />

// Radio: checked when state matches
<input type="radio" value="pro"
  checked={plan === 'pro'}
  onChange={e => setPlan(e.target.value)} />

// Select: value goes on <select>
<select value={city}
  onChange={e => setCity(e.target.value)}>
  <option value="paris">Paris</option>
</select>
```

> *Text-like = value. Toggle-like = checked.*

SB

TOPIC 7

Submitting Forms

One handler can serve every field.

Instead of one state variable per field, keep a single object and give each input a name. A single change handler updates the right key, and onSubmit sends everything together.

KEY POINTS

- One state object plus one handler covers all the fields
- The computed key [name]: value updates the matching field
- Put onSubmit on the form; preventDefault stops the reload
- A button with type="submit" also lets Enter submit
- To reset, set the state back to the initial object

```
const [form, setForm] = useState({ email: '', pass: '' });

const handleChange = (e) => {
  const { name, value } = e.target;
  setForm({ ...form, [name]: value });
};

const handleSubmit = (e) => {
  e.preventDefault();
  console.log(form);
};

<form onSubmit={handleSubmit}>
  <input name="email" value={form.email}
    onChange={handleChange} />
  <button type="submit">Send</button>
</form>
```

> *Give inputs a name, and one handler does the rest.*

TOPIC 8

LU

Lifting State Up

Share state through the nearest common parent.

When two components need the same data, move the state up to their closest common parent. The parent passes the value down as a prop and a setter down as a callback.

KEY POINTS

- If siblings need the same data, move it to their parent
- The value flows down as props; changes flow up as callbacks
- Children stay simple and reusable
- Data flows down, events flow up
- Do not lift higher than needed; Context handles the far cases

```
function Parent() {
  const [text, setText] = useState('');
  return (
    <>
      <Input text={text} onChange={setText} />
      <Preview text={text} />
    </>
  );
}

function Input({ text, onChange }) {
  return <input value={text}
    onChange={e => onChange(e.target.value)} />;
}

// Preview({ text }) just shows {text}
```

> *Data flows down. Events flow up.*

TOPIC 9

DR

Derived State

If you can calculate it, do not store it.

Many values can be worked out from state you already have: totals, counts, filtered lists. Calculate them during rendering instead of keeping a second copy that can drift out of sync.

KEY POINTS

- If it can be calculated from state or props, do not store it
- Duplicate state gets out of sync
- Calculating on every render is cheap in most cases
- Reach for useMemo only after measuring a real slowdown
- Keep state minimal: store the smallest set of facts

```
const [items, setItems] = useState([]);

// Bad: duplicated state, can desync
// const [count, setCount] = useState(0);
// setItems(...); setCount(items.length + 1);

// Good: calculate during render
const count = items.length;
const total = items.reduce((t, i) => t + i.price, 0);
const visible = items.filter(i => i.name.includes(q));

// Also avoid copying props into state
// const [name, setName] = useState(props.name);
// it will not follow later prop changes
```

> *Store the facts. Calculate the rest.*

TOPIC 10

VA

Validation & Hook Form

Check input before it leaves the form.

Validation makes sure the data is usable. You can write simple checks by hand and show error messages, or use React Hook Form to cut the boilerplate and keep re-renders low.

KEY POINTS

- Manual: check on submit or blur and keep messages in state
- Show each error next to its field
- React Hook Form: register connects inputs with few re-renders
- For larger forms add a Zod or Yup schema through a resolver
- Always validate on the server too; client checks are for UX only

```
// Manual validation
const errors = {};
if (!email.includes('@')) errors.email = 'Invalid email';
if (pass.length < 6) errors.pass = 'Min 6 characters';
{errors.email && <p className="err">{errors.email}</p>}

// React Hook Form
// npm install react-hook-form
const {
  register, handleSubmit,
  formState: { errors },
} = useForm();

<form onSubmit={handleSubmit(onSave)}>
  <input {...register('email', { required: true })} />
  {errors.email && <p>Email is required</p>}
</form>
```

> *Client validation helps users. Server validation protects you.*

QUICK REFERENCE

State & Forms Cheat Sheet

CONCEPT	PURPOSE	USE IT WHEN…
useState	Remember a value between renders	Counters, toggles, input text
setX(prev => ...)	Update from the latest value	New state depends on old state
Object state	Group related fields	Profiles, forms; copy with spread
Array state	Store lists	Add: spread, remove: filter, edit: map
Controlled input	React owns the input value	Live validation and formatting
checked vs value	Match the control type	Checkbox / radio vs text / select
Form submit	Collect and send data	onSubmit with preventDefault
Lifting state up	Share state between siblings	Two components need the same data
Derived state	Calculate instead of store	Totals, counts, filtered lists
Validation	Check data before sending	Manual checks or React Hook Form



Thank You

Questions about any topic? Let's discuss.

useState · Updater · Objects & arrays · Controlled inputs · Forms · Lifting state · Validation

NEXT: PART 5 — HOOKS (ALREADY COVERED IN YOUR LECTURE DECK)

REACT · LECTURE SERIES

React Hooks

A complete, in-depth, simple-English guide to every core Hook —
useState, useEffect, useContext, useRef, useMemo, useCallback, useReducer & custom Hooks.

Functional components • State & lifecycle • Best practices

LECTURE ROADMAP

What We'll Cover Today

S

useState

Store & update data in a component

E

useEffect

Run side effects (fetch, timers, subscriptions)

C

useContext

Share data without passing props manually

R

useRef

Persist values & reach real DOM elements

M

useMemo

Cache expensive calculations

CB

useCallback

Cache function definitions

RD

useReducer

Manage complex state logic

CH

Custom Hooks

Build your own reusable logic

THE BIG IDEA

What Exactly Is a Hook?

A Hook is just a special **JavaScript function** — always starting with the word **"use"** — that lets a simple function component "hook into" React features like state and lifecycle, which used to be available only in class components.

In one line:

> *"Hooks let function components remember things and react to changes — without ever writing a class."*

Why they were introduced (2019, React 16.8):

- Reuse stateful logic between components easily
- Avoid deeply nested "wrapper hell" from old patterns
- Keep related code together instead of split across lifecycle methods

BEFORE — Class Component

```
class Box extends React.Component {
  state = { count: 0 };
  render() {
    return <p>{this.state.count}</p>;
  }
}
```

AFTER — Function + Hook

```
function Box() {
  const [count] = useState(0);
  return <p>{count}</p>;
}
```

BEFORE YOU START

The Two Golden Rules of Hooks

1 Only call Hooks at the top level

Never put a Hook inside a loop, condition, or nested function. Always call Hooks in the exact same order on every single render, so React can correctly match state to the right Hook.

✓ DO

```
if (x) { ... }
const [v] = useState(0);
```

✗ DON'T

```
if (x) {
  const [v] = useState(0);
}
```

2 Only call Hooks from React functions

Call Hooks only from React function components, or from your own Custom Hooks. Never call them from a regular JavaScript function, a class, or an event handler directly.

✓ DO

```
function Profile() {
  const [n] = useState('');
}
```

✗ DON'T

```
function helper() {
  const [n] = useState('');
}
```

S

HOOK 1

useState

Give a function component its own memory.

useState lets a component store a value that can change over time — like a counter, form input, or toggle — and re-render automatically whenever that value updates.

KEY POINTS

- useState(initialValue) returns an array: [value, setterFunction]
- Calling the setter schedules a re-render with the new value
- State updates do NOT merge automatically like class state did
- Each useState call manages one independent piece of state
- Never mutate state directly — always call the setter function

```
import { useState } from 'react';


function Counter() {

  const [count, setCount] = useState(0);


  return (

    <button onClick={() => setCount(count + 1)}>

      Clicked {count} times

    </button>

  );
}
```

Simple way to remember: “box” = current value, “button” = the way to change it.

HOOK 2

useEffect

Run code in response to renders — side effects.

useEffect lets you run code that reaches outside React — fetching data, setting a timer, subscribing to an event — after the component renders, and clean it up when needed.

KEY POINTS

- Runs after the browser has painted the screen
- [] (empty array) = run once, only on mount
- [value] = re-run only when that value changes
- No array at all = run after every single render
- Return a function from it to clean up (timers, listeners)

```
import { useEffect, useState } from 'react';

function Clock() {

  const [time, setTime] = useState(new Date());

  useEffect(() => {

    const id = setInterval(() =>

      setTime(new Date()), 1000);

    return () => clearInterval(id); // cleanup

  }, []); // dependency array

  return <p>{time.toLocaleTimeString()}</p>;
}
```

The dependency array is React's way of asking: “when should this effect run again?”

C

HOOK 3

useContext

Share data across many components — no prop drilling.

useContext lets any component read a value from a shared “Context” directly, skipping the need to pass props down manually through every level in between.

KEY POINTS

- Removes the pain of passing props through many layers
- A Provider wraps components and supplies the shared value
- Any descendant can read it with useContext(TheContext)
- Great for themes, logged-in user, language, or app settings
- Updating the Provider's value re-renders every consumer

```
const ThemeContext = React.createContext('light');

function App() {

  return (

    <ThemeContext.Provider value='dark'>

      <Toolbar />

    </ThemeContext.Provider>

  );
}

function Toolbar() {

  const theme = useContext(ThemeContext);

  return <button className={theme}>Save</button>;
}
```

Think of Context like a radio station: any component can “tune in” without wiring.

HOOK 4

useRef

Keep a value around without causing a re-render.

useRef gives you a mutable “box” (.current) that survives across renders. Changing it never triggers a re-render, and it's also the standard way to directly access a DOM element.

KEY POINTS

- useRef(initialValue) returns an object: { current: value }
- Updating ref.current does NOT cause a re-render
- Attach it to a JSX tag with ref={...} to grab the DOM node
- Perfect for timers IDs, previous values, or focus control
- Different from state: refs are for values React shouldn't watch

```
import { useRef } from 'react';

function TextInput() {

  const inputRef = useRef(null);

  const focusInput = () => {

    inputRef.current.focus(); // DOM node

  };

  return (

    <>
      <input ref={inputRef} />

      <button onClick={focusInput}>Focus</button>
    </>
  );
}
```

Rule of thumb: if changing it should update the UI → useState. If not → useRef.

M

HOOK 5

useMemo

Skip expensive recalculations on every render.

useMemo remembers (“memoizes”) the result of a calculation, and only re-computes it when one of its listed dependencies actually changes, saving unnecessary work.

KEY POINTS

- Caches a computed VALUE between renders
- Recalculates only when a dependency in the array changes
- Useful for sorting, filtering, or heavy math on big lists
- Not free — it has its own small cost, so don't overuse it
- Only reach for it after noticing an actual performance issue

```
import { useMemo } from 'react';

function ProductList({ items, filter }) {

  const visible = useMemo(() => {

    console.log('filtering...');

    return items.filter(i =>

      i.category === filter);

  }, [items, filter]); // recompute if changed

  return visible.map(i =>
    <li key={i.id}>{i.name}</li>);
}
```

Simple test: “Is this calculation slow AND does it run on every render?” If yes, memoize it.

CB

HOOK 6

useCallback

Cache a function so it isn't rebuilt every render.

useCallback works just like useMemo, but for FUNCTIONS instead of values. It returns the same function reference across renders unless its dependencies change.

KEY POINTS

- Every render normally creates a brand-new function — useCallback prevents that
- Most useful when passing a function to a memoized child (React.memo)
- Without it, that child re-renders even when nothing meaningful changed
- useMemo caches a VALUE; useCallback caches a FUNCTION
- useCallback(fn, deps) is really just useMemo(() => fn, deps)

```
import { useCallback, useState } from 'react';

function Parent() {

  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {

    setCount(c => c + 1);

  }, []); // same function every render

  return <FastButton onClick={handleClick} />;
}

const FastButton = React.memo((
  { onClick }) => (
  <button onClick={onClick}>+1</button>
));
```

Pair it with React.memo — caching a function alone doesn't help unless a child depends on it.

HOOK 7

useReducer

A more organized way to manage complex state.

useReducer is an alternative to useState for state that's complex or involves multiple sub-values. You describe WHAT happened by dispatching an “action”, and a reducer function decides the next state.

KEY POINTS

- useReducer(reducerFn, initialState) returns [state, dispatch]
- dispatch(action) sends a plain object describing what happened
- The reducer is a pure function: (state, action) => newState
- Keeps state-update logic in one predictable, testable place
- Great choice when many actions can update related state fields

```
function reducer(state, action) {

  switch (action.type) {

    case 'inc': return { count: state.count + 1 };

    case 'dec': return { count: state.count - 1 };

    default: return state;
  } }

function Counter() {

  const [state, dispatch] =
    useReducer(reducer, { count: 0 });

  return (
    <button onClick={() => dispatch({type:'inc'})}>
      {state.count}
    </button>
  );
}
```

Same idea used by Redux! useReducer is essentially Redux's core pattern, built into React.

CH

HOOK 8

Custom Hooks

Package your own reusable logic as a Hook.

A Custom Hook is simply a JavaScript function whose name starts with “use”, and that calls other Hooks inside it. It lets you extract and reuse stateful logic across components.

KEY POINTS

- Solves the old problem of duplicating logic across components
- Built entirely from built-in Hooks like useState and useEffect
- Each component using it gets its own, fully independent state
- Naming it “useSomething” lets React enforce the Rules of Hooks
- Common examples: useFetch, useForm, useLocalStorage, useDebounce

```
// A reusable hook for window width

function useWindowWidth() {

  const [width, setWidth] =
    useState(window.innerWidth);

  useEffect(() => {

    const onResize = () =>
      setWidth(window.innerWidth);

    window.addEventListener('resize', onResize);

    return () =>
      window.removeEventListener('resize', onResize);

  }, []);
  return width;
}

const width = useWindowWidth(); // any component
```

Custom Hooks share LOGIC, not state — each caller gets a fresh, separate copy.

QUICK REFERENCE

Hooks Cheat Sheet

Hook	Purpose	Use It When...
`useState`	Store a piece of data that changes	You need simple local state
`useEffect`	Run side effects after render	Fetching data, timers, subscriptions
`useContext`	Read shared data from a Provider	Avoiding prop drilling
`useRef`	Persist a value / access a DOM node	Focus control, timers, previous value
`useMemo`	Cache a computed value	An expensive calculation is re-running
`useCallback`	Cache a function reference	Passing callbacks to memoized children
`useReducer`	Manage complex, multi-part state	Many related actions update one state



Thank You

Questions about any Hook? Let's discuss.

`useState · useEffect · useContext · useRef · useMemo · useCallback · useReducer · Custom Hooks`

REACT · LECTURE SERIES · PART 6

Routing

A complete, in-depth, simple-English guide to multi-page React apps with React Router v7 — routes, links, params, nested layouts, redirects, protected pages, lazy loading, loaders & actions.

React Router v7 • Layouts • Protected routes • Loaders

LECTURE ROADMAP

What We'll Cover Today

RT
Router Setup
createBrowserRouter and RouterProvider

LK
Link & NavLink
Navigate without reloading

ID
Dynamic Params
Read :id with useParams

OU
Nested Routes & Outlet
Layouts and child pages

NV
Navigating in Code
useNavigate and Navigate

SP
Search Params
Filters that live in the URL

404
404 & Error Pages
Catch-all and errorElement

PR
Protected Routes
Guard private pages

LZ
Lazy Routes
Load pages only when needed

LD
Loaders & Actions
Fetch and submit per route

THE BIG IDEA

What Is Client-Side Routing?

Routing decides which component to show for each URL. A React app is a single HTML page; React Router watches the address bar and swaps components, so moving between screens feels instant.

IN ONE LINE:

"The URL picks the component, and the page never reloads."

Why use a router:

- Instant page changes with no full reload
- Every screen gets its own shareable URL
- Back and Forward buttons just work
- Layouts and data can load per route

BEFORE — Multi-page Website

```
click link  ->  browser asks the server
server sends a NEW html page
the whole page reloads

// state, scroll position and speed lost
```

AFTER — React Router

```
click <Link>  ->  React Router
URL changes (History API)
only the matching component swaps

// no reload, state is kept
```

BEFORE YOU START

The Two Golden Rules of Routing

1 Use Link, not <a>, for internal pages

A normal anchor makes the browser reload the whole page and wipe your React state. Link changes the URL and swaps the component instead.

DO

```
<Link to="/about">About</Link>
```

DON'T

```
<a href="/about">About</a>
// full page reload, state lost
```

2 Put shared UI in a layout route

Write the navbar and footer once in a parent route that renders an Outlet, instead of copying them into every single page.

DO

```
{ element: <Layout />,
  children: [ /* pages */ ] }
// Layout renders <Outlet />
```

DON'T

```
<Navbar /> inside Home
<Navbar /> inside About
// copied into every page
```

TOPIC 1

RT

Router Setup

Describe your routes as an array of objects.

In React Router v7 you install one package, create a router from route objects, and hand it to RouterProvider. Each object maps a URL path to the element that should show.

KEY POINTS

- Routing maps a URL path to a component
- v7 needs only react-router; react-router-dom is the older name
- createBrowserRouter takes an array of { path, element } objects
- RouterProvider gives the router to your whole app
- The URL changes without a full page reload

```
# Install (React Router v7)
npm install react-router

// main.jsx
import { createRoot } from 'react-dom/client';
import { createBrowserRouter } from 'react-router';
import { RouterProvider } from 'react-router/dom';

const router = createBrowserRouter([
  { path: '/', element: <Home /> },
  { path: '/about', element: <About /> },
]);

createRoot(document.getElementById('root')).render(
  <RouterProvider router={router} />
);
```

> *URL in, component out.*

LK

TOPIC 2

Link & NavLink

Move between pages without reloading.

Link renders an anchor that changes the URL without a page reload. NavLink does the same and also knows whether it points to the page you are currently on, perfect for menus.

KEY POINTS

- Link renders an <a> but stops the full page reload
- A plain <a href> reloads the page and loses all state
- NavLink adds an active class automatically
- className and style can be functions of { isActive }
- Add the end prop to the "/" link so it is not always active

```
import { Link, NavLink } from 'react-router';

// Link: navigate without reload
<Link to="/about">About</Link>

// NavLink: knows if it is active
<NavLink to="/blog">Blog</NavLink>

// Style the active link
<NavLink
  to="/blog"
  className={({ isActive }) =>
    isActive ? 'link active' : 'link'}
>
  Blog
</NavLink>

// Never: <a href="/about"> = full reload
```

> *Link for moving. NavLink for menus that highlight.*

TOPIC 3

ID

Dynamic Params

One route, many pages: /posts/1, /posts/2.

A colon in the path, like :id, marks a dynamic segment. Whatever appears in that part of the URL is captured, and useParams hands it to your component.

KEY POINTS

- :id marks a dynamic segment of the path
- useParams returns an object with the captured values
- Values are always strings; convert with Number(id) if needed
- Use the param to fetch or find the right item
- A path can hold several params

```
// Route with a parameter
{ path: '/posts/:id', element: <Post /> }

// Link to it
<Link to={`/posts/${post.id}`}>Read</Link>

// Post.jsx: read the param
import { useParams } from 'react-router';

function Post() {
  const { id } = useParams();
  return <h1>Post number {id}</h1>;
}

// /posts/7  ->  id = '7'  (a string!)
// Several: /users/:userId/posts/:postId
```

Colon in the path, useParams in the component.

TOPIC 4

OU

Nested Routes & Outlet

Shared layouts with child pages inside.

Routes can have children. The parent route renders the shared layout, and an Outlet marks the spot where the matching child page appears. Write the navbar once and reuse it.

KEY POINTS

- Child routes render inside the parent's Outlet
- A layout route holds shared UI like navbar and footer
- index: true is the default child for the parent path
- Child paths are relative: no leading slash
- Nest as deep as your page structure needs

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <Layout />,
    children: [
      { index: true, element: <Home /> },
      { path: 'posts', element: <Posts /> },
      { path: 'posts/:id', element: <Post /> },
    ],
  },
]);

// Layout.jsx
import { Outlet } from 'react-router';
function Layout() {
  return (<><Navbar /><Outlet /><Footer /></>);
}
```

> *The Outlet is the empty slot where the child page appears.*

TOPIC 5

NV

Navigating in Code

Redirect after something happens.

Sometimes you need to change pages from code, for example after a successful login or save. useNavigate gives you a function for that, and Navigate does the same declaratively in JSX.

KEY POINTS

- useNavigate returns a function: navigate('/path')
- navigate(-1) goes back one step in history
- { replace: true } swaps the current history entry
- <Navigate> is the same redirect, written as a component
- These hooks only work inside components under the router

```
import { useNavigate, Navigate } from 'react-router';

function Login() {
  const navigate = useNavigate();

  const handleLogin = async () => {
    await login();
    navigate('/dashboard');
  };
}

// Replace the history entry (no Back to login)
navigate('/dashboard', { replace: true });

// Declarative redirect while rendering
if (!user) {
  return <Navigate to="/login" replace />;
}
```

> *After an action, navigate. While rendering, use <Navigate>.*

TOPIC 6

SP

Search Params

Keep filters and search in the URL.

The query string is the part after the question mark, like ?q=shoes&page=2. useSearchParams works like useState, but the value lives in the URL, so it survives refresh and can be shared.

KEY POINTS

- The query string is everything after the ? in the URL
- It works like useState, but the state is stored in the URL
- Refresh, share and the Back button all keep the filters
- Values are strings, so convert numbers yourself
- setParams replaces all params unless you merge them

```
import { useSearchParams } from 'react-router';

function Products() {
  const [params, setParams] = useSearchParams();
  const q = params.get('q') ?? '';

  return (
    <input
      value={q}
      onChange={e => setParams({ q: e.target.value })}
    />
  );
}

// URL: /products?q=shoes&page=2
params.get('page')   // '2'
// Filters become shareable and bookmarkable
```

> *If it should survive a refresh, put it in the URL.*

TOPIC 7

404

404 & Error Pages

Handle unknown URLs and crashes gracefully.

A path of * matches anything not matched earlier, which makes it your 404 page. An errorElement shows a fallback when a route throws an error while rendering or loading.

KEY POINTS

- path: '*' matches every URL nothing else matched: your 404
- errorElement is the fallback when a route throws
- useRouteError reads the error that was thrown
- Put errorElement on the root so a crash is never a blank page
- For response errors use isRouteErrorResponse(error)

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <Layout />,
    errorElement: <ErrorPage />,
    children: [ /* ... */ ],
  },
  // Catch-all: unknown URL
  { path: '*', element: <NotFound /> },
]);

// ErrorPage.jsx
import { useRouteError } from 'react-router';
function ErrorPage() {
  const error = useRouteError();
  return <p>Oops: {error.message}</p>;
}
```

> *Always have a catch-all and an error page.*

TOPIC 8

PR

Protected Routes

Keep private pages behind a login.

A protected route is a wrapper that checks whether the user is logged in. If yes, it shows the child page through an Outlet; if not, it redirects to the login page.

KEY POINTS

- A wrapper route checks login, then renders <Outlet /> or redirects
- Nest many pages as its children to guard them all at once
- replace stops Back from returning to a guarded page
- Role-based access: compare user.role before the Outlet
- This is UI only; the server must enforce permissions too

```
import { Navigate, Outlet } from 'react-router';

function ProtectedRoute() {
  const { user } = useAuth(); // your auth state
  if (!user) {
    return <Navigate to="/login" replace />;
  }
  return <Outlet />;
}

// In the router
{
  element: <ProtectedRoute />,
  children: [
    { path: '/dashboard', element: <Dashboard /> },
  ],
},
```

> *The router hides the page. The server protects the data.*

TOPIC 9

LZ

Lazy Routes

Download a page only when it is visited.

Code splitting breaks your app into chunks so visitors download only what they need. Loading each page lazily makes the first load smaller and faster.

KEY POINTS

- The page code downloads only on the first visit
- React.lazy plus Suspense is the general React tool
- The Suspense fallback shows while the chunk loads
- The route lazy property is the router's built-in version
- Split by route first: biggest win for the least effort

```
import { lazy, Suspense } from 'react';

const Admin = lazy(() => import('./pages/Admin'));

{
  path: '/admin',
  element: (
    <Suspense fallback={<p>Loading...</p>}>
      <Admin />
    </Suspense>
  ),
},

// Router-native: module exports Component
{ path: '/reports', lazy: () => import('./Reports') },
```

> *Do not ship the admin page to someone who never opens it.*

TOPIC 10

LD

Loaders & Actions

Fetch data and handle forms per route.

A loader runs before the page renders, so the component receives ready data. An action handles form submissions for the route. Both are available when you use createBrowserRouter.

KEY POINTS

- A loader runs before render, so data is ready on arrival
- useLoaderData reads what the loader returned
- No useEffect and no loading flag needed for the first load
- Actions with <Form> handle submits, then data revalidates
- Errors thrown in loaders go to the errorElement

```
// Loader: fetch BEFORE the page renders
{
  path: '/posts/:id',
  element: <Post />,
  loader: async ({ params }) => {
    const res = await fetch(`/api/posts/${params.id}`);
    return res.json();
  },
},

// Post.jsx
import { useLoaderData } from 'react-router';

function Post() {
  const post = useLoaderData();
  return <h1>{post.title}</h1>;
}
```

> *Fetch on the route, not inside the component.*

QUICK REFERENCE

Routing Cheat Sheet

CONCEPT	PURPOSE	USE IT WHEN…
createBrowserRouter	Define routes as objects	App entry point
RouterProvider	Give the router to React	main.jsx
Link / NavLink	Navigate without reload	Menus and links
useParams	Read :id from the URL	Detail pages
Outlet + children	Nested layouts	Shared navbar and footer
useNavigate	Redirect from code	After login or save
useSearchParams	Read and write ?query	Search, filters, pagination
errorElement + '*'	404 and crash pages	Unknown URL or a failure
Protected route	Guard private pages	Dashboard, profile, admin
loader / action	Fetch and submit per route	Data-driven pages



Thank You

Questions about any topic? Let's discuss.

Router setup · Link · Params · Nested routes · Navigate · Search params · Protected · Loaders

NEXT: PART 7 — DATA FETCHING & APIS

REACT · LECTURE SERIES · PART 7

Data & APIs

A complete, in-depth, simple-English guide to connecting React to real server data — REST, fetch, axios, loading states, CRUD, TanStack Query, RTK Query.

REST • fetch & axios • CRUD • TanStack Query • RTK Query



LECTURE ROADMAP

What We'll Cover Today

REST
HTTP & REST Basics
Methods, status codes and JSON

FE
The fetch API
Built-in requests and res.ok

AX
axios
A friendlier HTTP client

EF
useEffect Fetching Pattern
Fetch after render with state

UI
Loading, Error & Empty States
Show every outcome of a request

AB
AbortController & Race Conditions
Cancel outdated requests

CRUD
CRUD: POST, PUT, DELETE
Create, update and delete from the UI

INT
Axios Instance & Interceptors
One shared client with auth headers

TQ
TanStack Query
Cached server state and mutations

RTKQ
RTK Query Overview
Endpoints that become hooks

THE BIG IDEA

What Is Data Fetching?

Most real apps show data that lives on a server: posts, products, users. Your React app asks the server over HTTP, waits for the reply, then renders it. While it waits, the screen must still make sense.

IN ONE LINE:

"Server data is a cache: ask for it, wait, show it, keep it fresh."

Why it matters:

- Show real data instead of hard-coded arrays
- Handle waiting and failure, not just success
- Keep every screen in sync with the server
- Avoid duplicate requests with smart caching

BEFORE — Hard-coded data

```
const users = [
  { id: 1, name: "Sara" },
  { id: 2, name: "Ali" },
];
// never changes, never loads
```

AFTER — Server data

```
// ask the server, then render
fetch("/api/users")
  .then(res => res.json())
  .then(setUsers);
// plus loading, error, empty UI
```

BEFORE YOU START

The Two Golden Rules of Fetching

1 Handle loading, error and empty

A request can be waiting, failed or empty before it ever succeeds. Show something for each one, or users stare at a blank screen.

DO

```
if (loading) return <Spinner />;
if (error) return <p>{error}</p>;
if (!data.length) return <Empty />;
```

DON'T

```
return <List items={data} />;
// data is null at first: crash
// or blank screen on error
```

2 Fetch in an effect, never in render

Fetching in the body runs on every render, and setting state re-renders again, so you get an endless loop of requests.

DO

```
useEffect(() => {
  load();
}, []);
```

DON'T

```
function Users() {
  fetch(url).then(setData);
  // render, fetch, set, loop!
```

REST

TOPIC 1

HTTP & REST Basics

How your app talks to a server.

HTTP is the language browsers use to talk to servers. In a REST API each URL names a resource, and the method says what to do with it. The server replies with a status code and usually JSON.

KEY POINTS

- GET reads, POST creates, PUT and PATCH update, DELETE removes
- The URL names the thing; the method names the action
- 2xx means success, 4xx your mistake, 5xx server mistake
- Bodies and replies are usually JSON
- Practice for free on JSONPlaceholder (fake API)

```
// Method + URL = one action on one resource
GET    /posts        read the list
GET    /posts/1      read one post
POST   /posts        create a post
PUT    /posts/1      replace a post
PATCH  /posts/1      change some fields
DELETE /posts/1      remove a post

// Status codes
200 OK            201 Created
204 No Content    400 Bad Request
401 Unauthorized  404 Not Found
500 Server Error

// JSON body + header
Content-Type: application/json
{ "title": "Hello", "body": "Hi" }
```

> *Method is the verb, URL is the noun, status is the answer.*

FE

TOPIC 2

The fetch API

Built into the browser. No install needed.

fetch sends a request and returns a Promise. You turn the reply into data yourself with res.json(). It only rejects on a network failure, so a 404 or 500 still looks like success unless you check res.ok.

KEY POINTS

- fetch returns a Promise: await it or use .then
- You must call res.json() to get the data
- A 404 or 500 does NOT throw: check res.ok yourself
- For POST, set method, headers and a stringified body
- Only a network failure makes fetch reject

```
// GET
const res = await fetch(url);
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const data = await res.json();

// POST with a JSON body
const res = await fetch(url, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title: "Hi" }),
});
const created = await res.json();
```

> *fetch only fails on network errors. Always check res.ok.*

TOPIC 3

AX

axios

A friendlier HTTP client with less boilerplate.

axios is a small library that wraps HTTP requests. It parses JSON for you, throws on 4xx and 5xx status codes, and puts the data in res.data. Many teams like it for the shorter code.

KEY POINTS

- Install it with npm install axios
- The reply data lives in res.data (already parsed JSON)
- 4xx and 5xx responses throw, unlike fetch
- Objects in the body are sent as JSON automatically
- err.response.status holds the failing status code

```
# Install
npm install axios

import axios from 'axios';

// GET: data is already parsed
const { data } = await axios.get('/api/posts');

// POST: pass the object, no stringify
await axios.post('/api/posts', { title: 'Hi' });

// 4xx and 5xx throw automatically
try {
  await axios.get('/api/missing');
} catch (err) {
  console.log(err.response?.status); // 404
}
```

> *fetch is built in but manual. axios is a package with less typing.*

EF

TOPIC 4

useEffect Fetching Pattern

Fetch after render, then store the result.

Fetching is a side effect, so it belongs in useEffect, never in the render body. The usual pattern keeps three pieces of state: the data, a loading flag and an error. The effect runs after the first render.

KEY POINTS

- Fetch inside useEffect, never in the render body
- Keep three states: data, loading and error
- The effect callback cannot be async: declare a function inside
- [] runs once; list every variable the effect uses
- try / catch / finally makes sure loading always stops

```
function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function load() {
      try {
        const res = await fetch('/api/users');
        if (!res.ok) throw new Error(res.status);
        setUsers(await res.json());
      } catch (e) { setError(e.message); }
      finally { setLoading(false); }
    }
    load();
  }, []); // run once on mount
}
```

> *Render, then fetch, then set state, then render again.*

UI

TOPIC 5

Loading, Error & Empty States

Every request has four outcomes.

A request can be waiting, failed, empty or successful. Showing something for each one is what makes an app feel professional. Check them in order and return early.

KEY POINTS

- Four outcomes: loading, error, empty and success
- Check them in that order and return early
- Loading: a spinner or skeleton so the layout does not jump
- Error: a clear message plus a Retry button
- Empty is not an error: say "No results" kindly

```
function Users() {
  const { users, loading, error } = useUsers();

  if (loading) return <Spinner />;
  if (error) return <p className="err">{error}</p>;
  if (users.length === 0) {
    return <p>No users found.</p>;
  }

  // Only now is it safe to render the list
  return (
    <ul>
      {users.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}
```

> *A blank screen is a bug. Always say what is happening.*

AB

TOPIC 6

AbortController & Race Conditions

Cancel old requests so stale data never wins.

If the user types fast or leaves the page, an older request can finish after a newer one and overwrite fresh data. That is a race condition. Cancel the old request in the effect cleanup function.

KEY POINTS

- Race condition: an old reply arrives after a newer one
- Create an AbortController inside the effect
- Pass controller.signal to fetch (axios accepts signal too)
- Call abort() in the cleanup function
- Ignore AbortError in catch; it is not a real failure

```
useEffect(() => {
  const controller = new AbortController();

  async function load() {
    try {
      const res = await fetch(`/api/search?q=${q}`, {
        signal: controller.signal,
      });
      setResults(await res.json());
    } catch (e) {
      if (e.name !== 'AbortError') setError(e.message);
    }
  }
  load();

  return () => controller.abort(); // cleanup
}, [q]);
```

> *New query, cancel the old one. Cleanup does the cancelling.*

CRUD

TOPIC 7

CRUD: POST, PUT, DELETE

Create, update and delete from the UI.

After reading data, users want to change it. Each CRUD action is one HTTP method. Once the server answers, update local state with an immutable change so the screen matches. JSONPlaceholder fakes these writes, which is perfect for practice.

KEY POINTS

- Create is POST, update is PUT or PATCH, delete is DELETE
- Safe way: update state after the server replies OK
- Optimistic way: update first, roll back if it fails
- Never mutate state: use map, filter and spread
- Disable buttons while a request runs; show errors

```
// CREATE
const { data } = await axios.post('/posts', newPost);
setPosts(prev => [data, ...prev]);

// UPDATE (PUT replaces, PATCH changes fields)
const res = await axios.put(`/posts/${id}`, post);
setPosts(prev =>
  prev.map(p => (p.id === id ? res.data : p)));

// DELETE
await axios.delete(`/posts/${id}`);
setPosts(prev => prev.filter(p => p.id !== id));
```

> *Server first, then state. Keep the UI honest.*

INT

TOPIC 8

Axios Instance & Interceptors

One configured client for the whole app.

An instance stores the base URL, timeout and default headers once, so you stop repeating them. Interceptors are functions that run on every request or response, ideal for attaching a token in one place.

KEY POINTS

- axios.create() makes a client with shared settings
- Set baseURL, timeout and headers once
- Request interceptor: add a token to every request
- Response interceptor: catch a 401 and log the user out
- Import api everywhere instead of axios itself

```
// api.js
import axios from 'axios';

export const api = axios.create({
  baseURL: 'https://jsonplaceholder.typicode.com',
  timeout: 8000,
});

// Runs before every request
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

> *Configure once in api.js, use it everywhere.*

TOPIC 9

TQ

TanStack Query

Server state, cached and kept fresh for you.

TanStack Query (React Query v5) replaces the useEffect, loading and error boilerplate. You give each query a key and a fetch function; it caches the result, shares it between components and refetches when data goes stale.

KEY POINTS

- Install @tanstack/react-query and wrap the app in QueryClientProvider
- useQuery needs a queryKey and a queryFn
- The key is the cache: same key, shared data
- isPending means no data yet; error holds the failure
- After a mutation, invalidateQueries refetches fresh data

```
# npm install @tanstack/react-query
// main.jsx: wrap the app once
const queryClient = new QueryClient();
<QueryClientProvider client={queryClient}>
  <App />
</QueryClientProvider>
// Read: cached by its key
const { data, isPending, error } = useQuery({
  queryKey: ['posts'],
  queryFn: fetchPosts,
});

// Write, then refresh the list
const add = useMutation({
  mutationFn: createPost,
  onSuccess: () => queryClient.invalidateQueries({
    queryKey: ['posts'] }),
});
```

> *Key in, cached data out. Invalidate to refresh.*

RTKQ

TOPIC 10

RTK Query Overview

Data fetching built into Redux Toolkit.

RTK Query ships inside Redux Toolkit. You describe your endpoints once with createApi, and it generates hooks for loading, caching and refetching. Choose it when your app already uses Redux.

KEY POINTS

- Part of Redux Toolkit: no extra data library to install
- createApi describes all endpoints in one place
- It auto-generates hooks like useGetPostsQuery
- Add api.reducer and api.middleware to configureStore
- Tags (providesTags, invalidatesTags) refetch lists after changes

```
# npm install @reduxjs/toolkit react-redux
import { createApi, fetchBaseQuery }
  from '@reduxjs/toolkit/query/react';
export const api = createApi({
  baseQuery: fetchBaseQuery({
    baseUrl: 'https://jsonplaceholder.typicode.com',
  }),
  endpoints: builder => ({
    getPosts: builder.query({
      query: () => '/posts',
    }),
  }),
});
export const { useGetPostsQuery } = api;

// In a component
const { data, isLoading } = useGetPostsQuery();
```

> *Already on Redux? Use RTK Query. Otherwise TanStack Query is lighter.*

QUICK REFERENCE

Data Fetching Cheat Sheet

CONCEPT	PURPOSE	USE IT WHEN…
HTTP methods	GET, POST, PUT, PATCH, DELETE	Choosing the right action for a URL
fetch + res.ok	Built-in requests; check the status	Quick calls with no extra package
axios	Parsed JSON, throws on 4xx / 5xx	You want less boilerplate
useEffect + state	Fetch after render	Simple loads without a library
Loading / error / empty	Show every outcome	Any screen that shows server data
AbortController	Cancel outdated requests	Search boxes and fast page changes
POST / PUT / DELETE	Change data, then update state	Add, edit and remove items in the UI
axios.create	Shared baseURL, headers, interceptors	Many requests to one API, or auth tokens
useQuery / useMutation	Cached reads and writes	Most server data in a modern app
createApi (RTK Query)	Endpoints become hooks	Your app already uses Redux Toolkit



Thank You

Questions about any topic? Let's discuss.

HTTP · fetch · axios · States · Abort · CRUD · Interceptors · TanStack Query · RTK Query

NEXT: PART 8 — GLOBAL STATE MANAGEMENT

REACT · LECTURE SERIES · PART 8

Global State

A complete, in-depth, simple-English guide to sharing state across a whole React app — prop drilling, Context, useReducer, Redux Toolkit, Zustand, and how to choose.

Context • useReducer • Redux Toolkit • Zustand



LECTURE ROADMAP

What We'll Cover Today

PD
Prop Drilling
The pain that starts it all

CTX
Context API
Share a value with a subtree

PH
Provider + Custom Hook
One tidy way to share and read

RD
Context + useReducer
Organised updates through dispatch

RR
Context Re-render Pitfalls
Why one change renders too much

RTK
Redux Toolkit: Store & Slices
One store built from slices

SEL
useSelector & useDispatch
Read state, send actions

AT
createAsyncThunk
Async work inside Redux

ZU
Zustand
A tiny store with no Provider

VS
Choosing the Right Tool
Props, Context or a store?

THE BIG IDEA

What Is Global State?

Some data is needed by many components far apart: the logged-in user, the theme, the shopping cart. Passing it through every layer as props gets messy. Global state lives in one shared place that any component can read and update.

> IN ONE LINE:
>
> ***"Keep state close to where it is used, and share it only when you must."***

Why it matters:

- No more passing props through every layer
- One shared source of truth for the app
- Components update together automatically
- Updates become easier to track and debug

BEFORE — Prop drilling

```
<App user={user}>
  <Layout user={user}>
    <Navbar user={user}>
      <Avatar user={user} />
// 3 layers only pass it along
```

AFTER — Shared state

```
// any component reads it directly
const user = useUser();
<Avatar />   // no props needed
// and updates it from anywhere
```

BEFORE YOU START

The Two Golden Rules of Global State

1 Start local, go global only if needed

Props and useState are the simplest tools. Move state into Context or a store only when many distant components truly share it.

DO

```
// used by one component
const [open, setOpen] = useState(false);
```

DON'T

```
// app-wide store for one modal
dispatch(toggleModal());
```

2 Select only what you need

A component that reads the whole store re-renders on every change. Pick the small piece you use, and it re-renders only when that piece changes.

DO

```
const n = useCart(s => s.items.length);
```

DON'T

```
const cart = useCart();
// re-renders on ANY change
```

PD

TOPIC 1

Prop Drilling

Passing props through layers that do not use them.

Prop drilling means passing data through several components that only forward it, just to reach a deep child. It is fine for one or two levels, but it gets painful and fragile when the tree grows.

KEY POINTS

- Drilling = props passed through layers that never use them
- It is fine for one or two levels; do not over-react
- Many layers make code noisy and fragile to change
- Fix 1: composition, pass children or elements instead
- Fix 2: Context or a store, for truly shared data

```
// App -> Layout -> Sidebar -> Avatar
function App() {
  const [user, setUser] = useState({ name: "Sara" });
  return <Layout user={user} />;
}

function Layout({ user }) {
  // Layout does not use user, only passes it
  return <Sidebar user={user} />;
}

function Sidebar({ user }) {
  return <Avatar user={user} />;
}

function Avatar({ user }) {
  return <img alt={user.name} src={user.photo} />;
}
```

> *Prop drilling is a smell, not a bug. Fix it when it hurts.*

CTX

TOPIC 2

Context API

Share a value with a whole subtree.

Context lets a parent provide a value that any descendant can read directly, without props. You create it, wrap part of your tree in its Provider, and read it with useContext.

KEY POINTS

- createContext makes the context with a default value
- Provider supplies the value to everything inside it
- useContext reads the nearest Provider value
- No Provider above? You get the default value
- When the value changes, every reader re-renders

```
import { createContext, useContext } from 'react';
const ThemeContext = createContext("light");

// 1. Provide a value
function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Page />
    </ThemeContext.Provider>
  );
}

// 2. Read it anywhere below
function Button() {
  const theme = useContext(ThemeContext);
  return <button className={theme}>Save</button>;
}
```

> *Create, provide, consume. Three steps every time.*

PH

TOPIC 3

Provider + Custom Hook

Wrap Context in one clean, reusable pair.

In real apps, keep the state and the Provider together in one component, and export a custom hook to read it. Components then call a simple hook and never touch the raw context.

KEY POINTS

- Keep state and Provider together in one component
- Export a custom hook, like useAuth, to read it
- The hook can throw a clear error if there is no Provider
- Wrap the app once, near the root
- Components never import the raw context

```jsx
// AuthContext.jsx
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const value = { user, setUser };
  return (
    <AuthContext.Provider value={value}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("Wrap in <AuthProvider>");
  return ctx;
}
```

> *One Provider to wrap, one hook to read.*

RD

TOPIC 4

Context + useReducer

Predictable updates for bigger shared state.

When state has many fields or many kinds of updates, useState gets messy. A reducer puts every update in one function. Pair it with Context: the Provider holds state and dispatch, and any component can send actions.

KEY POINTS

- A reducer is a pure function: (state, action) gives new state
- dispatch sends an action like { type: "add", item }
- All update logic lives in one place, easy to test
- Provide both state and dispatch through Context
- Never mutate state in the reducer; return a new copy

```
function cartReducer(state, action) {
  switch (action.type) {
    case 'add': return [...state, action.item];
    case 'remove':
      return state.filter(i => i.id !== action.id);
    default:
      return state;
  }
}

export function CartProvider({ children }) {
  const [cart, dispatch] = useReducer(cartReducer, []);
  return (
    <CartContext.Provider value={{ cart, dispatch }}>
      {children}
    </CartContext.Provider>
  );
}
```

> *Context shares it. useReducer organises it.*

TOPIC 5

RR

Context Re-render Pitfalls

Why one change can re-render too much.

Every component that reads a context re-renders whenever the Provider value changes. A new object created during render counts as a change, so consumers re-render for no reason. Memoise the value and split big contexts.

KEY POINTS

- Every consumer re-renders when the Provider value changes
- An inline object value is new on every render
- Wrap the value in useMemo (and functions in useCallback)
- Split contexts: one for fast-changing, one for stable data
- Context has no selectors; a store can pick small pieces

```
// BAD: a new object on every render
<AppContext.Provider value={{ user, theme }}>

// BETTER: memoise the value
const value = useMemo(
  () => ({ user, setUser }),
  [user]
);

// BEST: split by how often things change
<UserContext.Provider value={user}>
  <ThemeContext.Provider value={theme}>
    {children}
  </ThemeContext.Provider>
</UserContext.Provider>

// Each consumer re-renders for its own context
```

> *Context is great for sharing, weak for fast-changing state.*

RTK

TOPIC 6

Redux Toolkit: Store & Slices

One store, many slices.

Redux keeps all global state in a single store. Redux Toolkit is the modern way to write it: a slice bundles a piece of state with the reducers that change it, and configureStore wires everything together.

KEY POINTS

- One store holds the whole app state
- A slice = state + reducers + generated actions
- Immer lets you "mutate" state safely inside reducers
- configureStore combines slices and enables DevTools
- Install both @reduxjs/toolkit and react-redux

```
# npm install @reduxjs/toolkit react-redux
// cartSlice.js
import { createSlice } from '@reduxjs/toolkit';
const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] },
  reducers: {
    addItem(state, action) {
      state.items.push(action.payload);
    },
  },
});
export const { addItem } = cartSlice.actions;
// store.js
import { configureStore } from '@reduxjs/toolkit';
export const store = configureStore({
  reducer: { cart: cartSlice.reducer },
});
```

> *Slice it, store it, provide it.*

SEL

TOPIC 7

useSelector & useDispatch

Read from the store, send actions to it.

Wrap the app in a Provider once. Then any component reads state with useSelector and updates it by calling dispatch with an action. A selector returns just the piece the component needs.

KEY POINTS

- Provider makes the store available to every component
- useSelector picks a piece of state: s => s.cart.items
- Select the smallest piece so re-renders stay rare
- useDispatch returns dispatch; call it with an action
- Action creators like addItem(product) build the action

```
// main.jsx: give the store to React
import { Provider } from 'react-redux';
<Provider store={store}>
  <App />
</Provider>

// CartBadge.jsx: read
import { useSelector } from 'react-redux';
const count = useSelector(s => s.cart.items.length);

// AddButton.jsx: write
import { useDispatch } from 'react-redux';
import { addItem } from './cartSlice';
const dispatch = useDispatch();
<button
  onClick={() => dispatch(addItem(product))}
>Add</button>
```

> *useSelector to read. useDispatch to write.*

TOPIC 8

AT

createAsyncThunk

Async work inside Redux.

A thunk runs async code, such as an API call, and then updates the store. createAsyncThunk generates three actions for you: pending, fulfilled and rejected. You handle them in extraReducers to track loading and errors.

KEY POINTS

- createAsyncThunk takes a type string and an async function
- It auto-creates pending, fulfilled and rejected actions
- Handle them in extraReducers with builder.addCase
- Dispatch it like any action: dispatch(getUsers())
- For plain server data, RTK Query or TanStack Query is simpler

```
import { createAsyncThunk } from '@reduxjs/toolkit';

export const getUsers = createAsyncThunk(
  'users/fetch',
  async () => {
    const res = await fetch('/api/users');
    return res.json();
  }
);

// inside createSlice
extraReducers: builder => {
  builder
    .addCase(getUsers.pending, s => { s.loading = true; })
    .addCase(getUsers.fulfilled, (s, a) => {
      s.list = a.payload; s.loading = false;
    });
},
```

> *pending, fulfilled, rejected: the three lives of a request.*

ZU

TOPIC 9

Zustand

A tiny store with almost no setup.

Zustand is a small state library. You create a store with one function and use it as a hook. There is no Provider, no reducer and no action to write, and components re-render only for the state they select.

KEY POINTS

- Install with npm install zustand; import { create }
- create() returns a hook; state and actions live together
- No Provider and no reducer needed
- Pass a selector to re-render only for that piece
- set merges your update into the current state

```
# npm install zustand
import { create } from 'zustand';

const useCart = create(set => ({
  items: [],
  add: i => set(s => ({ items: [...s.items, i] })),
  clear: () => set({ items: [] }),
}));

// Use it in any component, no Provider
const items = useCart(s => s.items);
const add = useCart(s => s.add);
<button onClick={() => add(product)}>Add</button>

// Read outside React too
useCart.getState().items
```

> *A whole store in one function. Select what you need.*

TOPIC 10

VS

Choosing the Right Tool

Pick the simplest tool that works.

No tool is best everywhere. Ask where the state belongs and how often it changes. Start with props, move to Context for small shared values, and use a store when many components update shared data often.

KEY POINTS

- Start simple: useState, then props, then Context, then a store
- Context fits values that change rarely: theme, user, language
- Stores fit shared state that changes often
- Redux Toolkit suits big teams; Zustand suits small apps
- Server data belongs in a query library, not a global store

```
// Ask in this order

1. Used by one component?
   -> useState
2. Shared by a few nearby components?
   -> lift state up, pass props
3. Read by many, changes rarely?
   -> Context (theme, user, language)
4. Shared, updated often, complex?
   -> Zustand or Redux Toolkit
5. Data that comes from a server?
   -> TanStack Query or RTK Query

// Redux Toolkit: big teams, strict structure
// Zustand: small apps, minimum code
```

> *Simplest tool first. Upgrade only when it hurts.*

QUICK REFERENCE

Global State Cheat Sheet

CONCEPT	PURPOSE	USE IT WHEN…
Prop drilling	Pass props through layers	Only a level or two separates the data
createContext / useContext	Share a value with a subtree	Theme, user or language needed in many places
Provider + custom hook	One clean pair to share and read	You want a tidy API like useAuth()
Context + useReducer	Organised updates through dispatch	Shared state with many kinds of updates
Memoised / split context	Avoid needless re-renders	A Provider value is an object or function
createSlice + configureStore	State, reducers and actions in one	Setting up Redux Toolkit state
useSelector / useDispatch	Read state, send actions	Components that use a Redux store
createAsyncThunk	Async calls with 3 lifecycle actions	Loading API data into a Redux slice
Zustand create()	A small hook-based store	You want global state with minimal code
Choosing a tool	Match the tool to the problem	Starting a feature, unsure where state lives



Thank You

Questions about any topic? Let's discuss.

Prop drilling · Context · useReducer · Re-renders · Redux Toolkit · Thunks · Zustand · Choosing

NEXT: PART 9 — PERFORMANCE & ADVANCED PATTERNS

REACT · LECTURE SERIES · PART 9

Performance

A complete, in-depth, simple-English guide to fast React apps — rendering, keys, memo, virtualization, code splitting, transitions, error boundaries, portals and patterns.

Rendering • memo • Virtualization • Suspense • Patterns

LECTURE ROADMAP

What We'll Cover Today

RN
How Rendering Works
Trigger, render and commit

KY
Keys & Reconciliation
How React decides what changed

MEM
React.memo
Skip renders when props match

UM
useMemo & useCallback in Practice
Cache values and functions wisely

VL
List Virtualization
Render only the visible rows

CS
Code Splitting & Suspense
Load code only when needed

TR
Transitions (useTransition)
Keep the UI responsive

EB
Error Boundaries
Catch crashes with a fallback

PT
Portals & Modals
Render outside the parent

PAT
Compound, HOC & Render Props
Reusable component patterns

THE BIG IDEA

What Makes React Slow?

React is fast by default, but every state change re-renders a component and all of its children. Slow screens usually come from too many renders, huge lists, or a big JavaScript bundle. Knowing which one you have tells you which fix to use.

> IN ONE LINE:
>
> ***"Fast apps come from fewer, cheaper renders, not from memo everywhere."***

Why it matters:

- Find the real bottleneck instead of guessing
- Render less work: memo, keys, virtualization
- Ship less code: lazy loading and splitting
- Keep the UI responsive with transitions

BEFORE — Everything re-renders

```
setCount(count + 1)
  -> App renders
  -> Header, List, Footer render
  -> 10,000 rows render
// slow, laggy typing
```

AFTER — Targeted work

```
setCount(count + 1)
  -> App renders
  -> memo skips List and Footer
  -> only the counter updates
// fast, smooth typing
```

BEFORE YOU START

The Two Golden Rules of Performance

1 Measure first, optimise second

Most React apps are already fast enough. Use the Profiler to find the one slow component, fix it, then measure again to prove it helped.

DO

```
// 1. Profile and find the slow part
// 2. Fix one thing
// 3. Profile again and compare
```

DON'T

```
// wrap everything in memo
// "just in case", with no proof
```

2 Keep render pure

A component must give the same output for the same props and state. Calculate during render; never change anything outside it.

DO

```
const sorted = [...items].sort();
// calculate, change nothing
```

DON'T

```
items.sort();   // mutates the prop
window.hits += 1;  // outside change
```

TOPIC 1

RN

How Rendering Works

Trigger, render, commit.

React updates the screen in three steps: a trigger (state or props change), a render (React calls your components to work out the new UI), and a commit (React applies only the differences to the DOM). Rendering is not the same as touching the DOM.

KEY POINTS

- Three steps: trigger, render, commit
- A state change or a parent re-render triggers a render
- Render means React calls your components; it is not the DOM
- By default a re-render also re-renders all children
- Commit touches the DOM only where something changed

```
function Counter() {
  const [n, setN] = useState(0);
  console.log("render");
  return (
    <>
      <button onClick={() => setN(n + 1)}>{n}</button>
      <Child />
    </>
  );
}

// 1. TRIGGER: setN is called
// 2. RENDER: Counter and Child both run again
// 3. COMMIT: only changed DOM nodes update

// A render is not a DOM update.
```

> *A render is just a function call. The commit is the DOM work.*

KY

TOPIC 2

Keys & Reconciliation

How React decides what changed.

Reconciliation compares the new tree with the old one and updates only the differences. In lists, the key tells React which item is which. Stable keys keep items and their state in the right place when the list changes.

KEY POINTS

- Reconciliation: React diffs the old and new trees
- A different element type makes React rebuild that subtree
- A key must be stable and unique among siblings
- Avoid the index if items can move, be added or deleted
- Changing a key forces a brand-new component with fresh state

```
// GOOD: a stable, unique id
{users.map(u => (
  <User key={u.id} user={u} />
))}

// BAD: index as key in a changing list
{users.map((u, i) => (
  <User key={i} user={u} />
))}
// delete the first item: state shifts to the wrong row

// Trick: a new key resets a component
<Profile key={userId} id={userId} />
// userId changes -> fresh state, nothing stale
```

> *Same key, same item. New key, new item.*

MEM

TOPIC 3

React.memo

Skip re-renders when props did not change.

By default a child re-renders whenever its parent does. React.memo wraps a component so React skips it when its props are shallow-equal to last time. It helps only when the component is slow and its props stay the same.

KEY POINTS

- memo skips a re-render if the props are shallow-equal
- Typing in the input re-renders List, but Row is skipped
- New object, array or function props break memo
- Use it for slow components or long lists, not everywhere
- The React Compiler can add this memoization for you

```
import { memo, useState } from 'react';

const Row = memo(function Row({ user }) {
  console.log("Row render", user.id);
  return <li>{user.name}</li>;
});

function List({ users }) {
  const [q, setQ] = useState("");
  return (
    <>
      <input value={q}
        onChange={e => setQ(e.target.value)} />
      {users.map(u => <Row key={u.id} user={u} />)}
    </>
  );
}
```

> *memo is a promise: same props, same output.*

UM

TOPIC 4

useMemo & useCallback in Practice

Remember values and functions between renders.

useMemo caches a calculated value, and useCallback caches a function, until their dependencies change. Their best use is keeping props stable for memoised children, or skipping a truly expensive calculation.

KEY POINTS

- useMemo caches a value; useCallback caches a function
- Both re-run only when a dependency changes
- Memoising cheap work costs more than it saves
- Most useful to keep props stable for memo children
- With the React Compiler you often need them far less

```
// Cache an expensive calculation
const visible = useMemo(
  () => filterBig(items, query),
  [items, query]
);

// Cache a function so memo(Row) can skip
const onSelect = useCallback(
  id => setSelected(id),
  []
);

<Row key={u.id} user={u} onSelect={onSelect} />

// Without useCallback, onSelect is new each render
// and memo(Row) re-renders anyway
```

> *Measure the cost first. Memoising has a cost too.*

TOPIC 5

VL

List Virtualization

Render only the rows you can see.

A list of 10,000 rows creates 10,000 DOM nodes and feels slow. Virtualization (windowing) renders only the rows visible in the viewport, plus a few extra. The react-window library does this for you.

KEY POINTS

- Only the visible rows exist in the DOM
- Great for lists with hundreds or thousands of rows
- react-window v2: List with rowComponent, rowCount, rowHeight
- Pass the style prop to each row so it is positioned
- Give the list a fixed height; it scrolls inside it

```
# npm install react-window
import { List } from 'react-window';

function Row({ index, style, users }) {
  return <div style={style}>{users[index].name}</div>;
}

function BigList({ users }) {
  return (
    <List
      rowComponent={Row}
      rowCount={users.length}
      rowHeight={36}
      rowProps={{ users }}
      style={{ height: 400 }}
    />
  );
}
```

> *Do not render what nobody can see.*

TOPIC 6

CS

Code Splitting & Suspense

Download code only when it is needed.

A big bundle slows the first load. Code splitting breaks it into chunks. React.lazy loads a component on demand, and Suspense shows a fallback while the chunk downloads.

KEY POINTS

- lazy() takes a function that returns a dynamic import()
- The lazy component loads the first time it renders
- Suspense shows a fallback while it loads
- The module must have a default export
- Good targets: routes, charts, editors and modals

```
import { lazy, Suspense } from 'react';

// Not in the main bundle
const Chart = lazy(() => import('./Chart'));

function Dashboard() {
  return (
    <>
      <h1>Sales</h1>
      <Suspense fallback={<p>Loading chart...</p>}>
        <Chart />
      </Suspense>
    </>
  );
}

// Chart.jsx must have a default export
```

> *Ship less code on the first load.*

TOPIC 7

TR

Transitions (useTransition)

Keep typing smooth while heavy updates run.

Some updates are urgent, like typing, and some can wait, like filtering a huge list. startTransition marks the slow update as non-urgent, so React keeps the input responsive and can interrupt the slow render.

KEY POINTS

- Urgent updates, like typing, stay instant
- Updates inside startTransition are low priority
- isPending lets you show a subtle loading hint
- React can interrupt and restart the slow render
- It does not make code faster; it only reorders the work

```
import { useState, useTransition } from 'react';

function Search({ items }) {
  const [q, setQ] = useState("");
  const [shown, setShown] = useState(items);
  const [isPending, startTransition] = useTransition();

  function onChange(e) {
    setQ(e.target.value);              // urgent
    startTransition(() => {
      setShown(filterBig(items, e.target.value));
    });                                // can wait
  }
  // isPending is true while the slow update runs
}
```

> *Same work, better order. Typing always wins.*

TOPIC 8

EB

Error Boundaries

Catch crashes so one bug does not blank the app.

If a component throws while rendering, React unmounts the whole tree. An error boundary catches the error and shows a fallback instead. Boundaries are still written as class components; the react-error-boundary package wraps this for you.

KEY POINTS

- A render error with no boundary unmounts the whole app
- A boundary shows a fallback UI instead
- It must be a class with getDerivedStateFromError
- It does NOT catch event handler or async errors
- react-error-boundary offers a ready-made version

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    console.error(error, info);   // log it
  }

  render() {
    if (this.state.hasError) return <p>Oops!</p>;
    return this.props.children;
  }
}

<ErrorBoundary><Widget /></ErrorBoundary>
```

> *Wrap risky parts so one widget cannot kill the page.*

TOPIC 9

PT

Portals & Modals

Render outside the parent, keep React in control.

A portal renders a component into a different DOM node, such as document.body, while it stays in the same place in the React tree. It is the standard way to build modals and tooltips that escape overflow and z-index limits.

KEY POINTS

- createPortal(children, domNode) renders elsewhere in the DOM
- The component stays in the same place in the React tree
- It escapes overflow: hidden and z-index traps
- Events still bubble through the React tree
- Make it accessible: role="dialog", Escape key and focus

```
import { createPortal } from 'react-dom';

function Modal({ open, onClose, children }) {
  if (!open) return null;

  return createPortal(
    <div className="overlay" onClick={onClose}>
      <div onClick={e => e.stopPropagation()}>
        {children}
      </div>
    </div>,
    document.body
  );
}

// Events still bubble to React parents
```

> *Rendered anywhere in the DOM, still a child in React.*

PAT

TOPIC 10

Compound, HOC & Render Props

Patterns for flexible, reusable components.

Compound components share hidden state through Context so their parts work together, like Tabs and Tab. A higher-order component (HOC) wraps a component to add behaviour. A render prop is a function prop that decides what to draw.

KEY POINTS

- Compound: the parent holds state, children share it via Context
- HOC: a function that takes a component and returns a better one
- Render prop: a prop that is a function returning UI
- Custom hooks now replace most HOCs and render props
- React 19: ref is a normal prop, so forwardRef is rarely needed

```
// 1. Compound: parts share state via Context
function Tabs({ children }) {
  const [tab, setTab] = useState(0);
  return (
    <TabsCtx.Provider value={{ tab, setTab }}>
      {children}
    </TabsCtx.Provider>
  );
}
<Tabs><Tab index={0}>Info</Tab></Tabs>

// 2. HOC: a function that wraps a component
const withUser = Comp => props =>
  <Comp {...props} user={useUser()} />;

// 3. Render prop: a function decides the UI
<Mouse render={pos => <p>{pos.x}</p>} />
```

> *Share logic with hooks. Share structure with compound components.*

QUICK REFERENCE

Performance Cheat Sheet

CONCEPT	PURPOSE	USE IT WHEN…
Trigger, render, commit	How React updates the screen	Explaining why a component re-rendered
key	Identity of list items	Rendering lists or forcing a reset
React.memo	Skip a render if props are equal	A slow child re-renders for no reason
useMemo / useCallback	Cache a value or a function	Slow calculations or stable props for memo
react-window	Render only the visible rows	Lists with hundreds or thousands of rows
lazy + Suspense	Load code on demand	Heavy routes, charts and editors
useTransition	Mark an update as non-urgent	A slow update makes typing laggy
Error boundary	Catch render errors, show a fallback	Around risky widgets and pages
createPortal	Render outside the parent DOM	Modals, tooltips and dropdowns
Component patterns	Reusable ways to share logic and UI	Flexible APIs like Tabs and Accordion



Thank You

Questions about any topic? Let's discuss.

Rendering · Keys · memo · useMemo · Virtualization · Lazy · Transitions · Boundaries · Portals · Patterns

NEXT: PART 10 — AUTH, TESTING & DEPLOYMENT

REACT · LECTURE SERIES · PART 10

Ship It

A complete, in-depth, simple-English guide to finishing a React app — authentication, JWT, role-based access, testing with Vitest and MSW, TypeScript, builds and deployment.

Auth • Vitest & RTL • MSW • TypeScript • Deploy



LECTURE ROADMAP

What We'll Cover Today

AU
Authentication Flow
Login, token and logout

JWT
JWT & Cookies
Where the proof of login lives

RB
Role-based Access
Guard pages by user role

VT
Vitest Setup
A fast test runner for Vite

RTL
React Testing Library
Test what the user sees

MSW
Mocking APIs with MSW
Fake the server in tests

TS
TypeScript: Props & State
Typed components

TS2
TypeScript: Hooks & Events
Typed events, refs and generics

ENV
Production Build & Env Vars
Optimised files and config

GO
Deploying (Vercel / Netlify) & Next Steps
Go live, then keep growing

THE BIG IDEA

What Makes an App Production-Ready?

A demo that works on your laptop is not a finished product. A real app lets users log in safely, has tests that catch bugs before users do, and runs on a public URL. This part covers those last steps.

> IN ONE LINE:
>
> ***"Secure it, test it, type it, ship it."***

Why it matters:

- Users and their data stay protected
- Tests catch bugs before your users do
- Types catch mistakes while you type
- A live URL you can show to anyone

BEFORE — Works on my laptop

```
npm run dev
// no login: anyone sees any page
// no tests: users find the bugs
// plain JS: typos found at runtime
// only runs on localhost
```

AFTER — Ready to ship

```
// login and roles guard the pages
// tests run on every change
// TypeScript checks the code
npm run build  ->  deploy
// live at https://my-app.vercel.app
```

BEFORE YOU START

The Two Golden Rules of Shipping

1 Never trust the browser

Anyone can edit the page, read your JavaScript and fake requests. Hiding a button is a courtesy; the server must check every request.

DO

```
// the server checks token and role
if (!user.isAdmin) return 403;
```

DON'T

```
// only hiding the button
{isAdmin && <DeleteButton />}
// not security on its own
```

2 Test what the user sees

Find elements the way a user does, and check what appears on screen. Tests tied to internals break every time you refactor.

DO

```
screen.getByRole('button', { name: /save/i })
```

DON'T

```
expect(component.state.count).toBe(1);
// testing internals, breaks on refactor
```

AU

TOPIC 1

Authentication Flow

Prove who the user is, then remember it.

Authentication answers one question: who are you? The user sends an email and password, the server checks them and replies with a token, and the app stores it and sends it with later requests. Logging out simply removes it.

KEY POINTS

- Login: send credentials, receive a token or a session
- Store the user in app state (Context or a store)
- Send the token with every request, in one interceptor
- Logout clears the token and the user state
- Authentication = who you are. Authorization = what you may do

```
// 1. Send credentials to the server
const { data } = await api.post('/auth/login', {
  email, password,
});

// 2. Keep the result in app state
setUser(data.user);
localStorage.setItem('token', data.token);

// 3. Send it on every request (interceptor)
config.headers.Authorization = `Bearer ${token}`;

// 4. Logout: forget everything
function logout() {
  localStorage.removeItem('token');
  setUser(null);
}
```

> *Log in, store, send, log out. Four steps.*

JWT

TOPIC 2

JWT & Cookies

Where the proof of login lives.

A JWT (JSON Web Token) is a signed string the server gives you after login. It carries claims such as your user id and an expiry time. You can keep it in localStorage or in an httpOnly cookie, and each choice has security trade-offs.

KEY POINTS

- A JWT is header.payload.signature, signed by the server
- The payload is readable by anyone: never put secrets in it
- localStorage is simple, but any XSS bug can steal the token
- An httpOnly cookie cannot be read by JavaScript
- Tokens expire; a refresh token quietly gets a new one

```
// A JWT has 3 parts: header.payload.signature
eyJhbGciOi.eyJzdWIiOiI0MiJ9.k3Jf9x

// The payload is only encoded, NOT secret
const payload = JSON.parse(atob(token.split(".")[1]));
// { sub: "42", role: "admin", exp: 1893456000 }

// Option A: localStorage (JS can read it)
localStorage.setItem('token', token);

// Option B: httpOnly cookie, set by the server
// JS cannot read it, so XSS cannot steal it
api.defaults.withCredentials = true;

// Expired? Check exp before trusting it
const expired = payload.exp * 1000 < Date.now();
```

> *A JWT proves identity. It does not hide data.*

RB

TOPIC 3

Role-based Access

Different users, different pages.

Authorization decides what a logged-in user may do. Give each user a role, like user or admin, and check it in a route guard. This only controls the UI; the server must check the role on every request too.

KEY POINTS

- Authentication = who you are; authorization = what you may do
- Store a role on the user: user, admin, teacher…
- A guard route checks login first, then the role
- Hiding a button is not security
- The server must check the role on every request

```
import { Navigate, Outlet } from 'react-router';

function RoleRoute({ allow }) {
  const { user } = useAuth();

  if (!user) {
    return <Navigate to="/login" replace />;
  }
  if (!allow.includes(user.role)) {
    return <Navigate to="/403" replace />;
  }
  return <Outlet />;
}

// In the router
{ element: <RoleRoute allow={['admin']} />,
  children: [{ path: '/admin', element: <Admin /> }] }
```

> *The UI is a courtesy. The server is the lock.*

TOPIC 4

VT

Vitest Setup

A fast test runner built for Vite.

Vitest runs your tests with the same Vite config as your app, so imports and JSX just work. Install it with jsdom (a fake browser) and Testing Library, then point Vitest at a setup file.

KEY POINTS

- Vitest reuses your Vite config: no extra Babel setup
- jsdom simulates a browser DOM inside Node
- setupFiles runs before every test file
- jest-dom adds matchers like toBeInTheDocument()
- Run npm test; it re-runs affected tests as you save

```
# npm i -D vitest jsdom @testing-library/react
#   @testing-library/dom @testing-library/jest-dom
#   @testing-library/user-event

// vite.config.js
import { defineConfig } from 'vitest/config';
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    setupFiles: './src/setupTests.js',
  },
});

// src/setupTests.js
import '@testing-library/jest-dom/vitest';

// package.json scripts: "test": "vitest"
```

> *One toolchain for building and testing.*

RTL

TOPIC 5

React Testing Library

Test what the user sees and does.

React Testing Library renders a component into jsdom and lets you find elements the way a user would: by role, label or text. Then you click, type and check the result. It never looks at state or internal code.

KEY POINTS

- render() puts the component into a fake DOM
- Query like a user: getByRole, getByLabelText, getByText
- userEvent simulates real clicks and typing
- Use await with user events and findBy queries
- Never test state or internal functions

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { test, expect } from 'vitest';

test('counter adds one on click', async () => {
  render(<Counter />);
  const user = userEvent.setup();

  // Find it like a user would
  const btn = screen.getByRole('button', { name: /add/ });
  await user.click(btn);

  const text = screen.getByText("Count: 1");
  expect(text).toBeInTheDocument();
});
```

> *If a user could not notice it, do not test it.*

MSW

TOPIC 6

Mocking APIs with MSW

Fake the server, keep your real fetch code.

Tests should not hit a real server. MSW (Mock Service Worker) intercepts network requests inside the test and returns the responses you define, so your fetch or axios code runs unchanged. The v2 API uses http and HttpResponse.

KEY POINTS

- Install with npm install -D msw (v2 API)
- Handlers use http.get / http.post and HttpResponse.json
- setupServer comes from msw/node, for tests
- resetHandlers after each test keeps tests independent
- Override a handler with server.use to test errors

```
// src/mocks/server.js
import { http, HttpResponse } from 'msw';
import { setupServer } from 'msw/node';

export const server = setupServer(
  http.get('https://api.example.com/users', () =>
    HttpResponse.json([{ id: 1, name: "Sara" }])
  )
);

// setupTests.js
beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());

// In one test: simulate a server error
server.use(http.get('https://api.example.com/users',
  () => new HttpResponse(null, { status: 500 })));
```

> *Real code, fake server. Tests stay honest.*

TOPIC 7

TS

TypeScript: Props & State

Let the compiler catch mistakes for you.

TypeScript adds types to JavaScript. In React you type props with an interface or type, and you give useState a type when it cannot be guessed from the initial value. Create a project with the react-ts template.

KEY POINTS

- Create a project with --template react-ts
- Describe props with an interface or a type
- Optional props use ?, unions use |
- useState<Type> when the initial value hides the type
- Function props get full signatures: (id: number) => void

```
# npm create vite@latest app -- --template react-ts

interface User {
  id: number;
  name: string;
  role?: "admin" | "user";   // optional
}

interface CardProps {
  user: User;
  onSelect: (id: number) => void;
}

function Card({ user, onSelect }: CardProps) {
  // state that starts empty needs a type
  const [sel, setSel] = useState<User | null>(null);
  return <button onClick={() => onSelect(user.id)} />;
}
```

> *Props in, types out. The compiler checks both ends.*

TOPIC 8

TS2

TypeScript: Hooks & Events

Type events, refs, generics and actions.

Event handlers, refs and reducers all benefit from types. React ships helper types such as ChangeEvent, and useRef needs the element type. Generics let one component work with many item types.

KEY POINTS

- ChangeEvent<HTMLInputElement> types an input handler
- useRef<HTMLInputElement>(null) types a DOM ref
- Generics like <T> make one component fit many types
- Union types describe reducer actions safely
- Set "strict": true in tsconfig to catch the most mistakes

```
import type { ChangeEvent } from 'react';

function onChange(e: ChangeEvent<HTMLInputElement>) {
  setName(e.target.value);
}

// Refs: element type + null
const box = useRef<HTMLInputElement>(null);
box.current?.focus();

// Generic component: works for any item type
function List<T>({ items }: { items: T[] }) {
  return <ul>{items.length}</ul>;
}

// useReducer: type the action
type Action = { type: 'inc' } | { type: 'dec' };
```

> *Type the edges: props, events and API data.*

ENV

TOPIC 9

Production Build & Env Vars

Turn your code into fast, small files.

The dev server is for development only. npm run build bundles, minifies and splits your code into a dist folder. Settings that change between environments, like the API URL, belong in environment variables, never in the code.

KEY POINTS

- npm run build creates optimised files in dist/
- npm run preview tests the production build locally
- Vite exposes only variables that start with VITE_
- Anything in the browser can be read: no secret keys
- Add .env files to .gitignore; commit a .env.example

```
# Build for production, then test it locally
npm run build      # writes the dist/ folder
npm run preview    # serves dist/ on your machine

# .env (not committed)
VITE_API_URL=https://api.example.com

// Read it in code
const url = import.meta.env.VITE_API_URL;

// Only VITE_ variables reach the browser
// Everything in the browser is public:
// never put secret keys in a Vite env var

# .gitignore
.env
.env.local
```

> *Secrets belong on the server. The browser sees everything.*

TOPIC 10

GO

Deploying (Vercel / Netlify) & Next Steps

Put your app on a public URL.

Push your project to GitHub, import it into Vercel or Netlify, and every push deploys automatically. Set the build command, the output folder and your VITE_ variables. Single-page apps need a rewrite so refreshing a route does not give a 404.

KEY POINTS

- Push to GitHub; the host deploys on every push
- Build command npm run build, output directory dist
- Add VITE_ variables in the host dashboard, then redeploy
- SPAs need a rewrite to index.html, or refresh gives 404
- Next: Next.js for SSR and React Server Components

```
# Both platforms: connect your GitHub repo
Build command:    npm run build
Output directory: dist
Env variables:    VITE_API_URL=...

// vercel.json: fix 404 on refresh
{ "rewrites": [
  { "source": "/(.*)", "destination": "/index.html" }
] }

# Netlify: public/_redirects
/*  /index.html  200

// Next steps: Next.js (SSR, routing, RSC),
// then full stack: Node, Express, a database
```

> *A live URL in your portfolio beats code on your laptop.*

QUICK REFERENCE

Ship It Cheat Sheet

CONCEPT	PURPOSE	USE IT WHEN…
Login flow	Send credentials, store the token	Letting users sign in and out
JWT / cookies	Signed proof of login	Keeping a session across requests
RoleRoute	Guard pages by user role	Separate admin and user areas
Vitest + jsdom	Fast test runner with a fake DOM	Running tests in a Vite project
render + screen	Find elements like a user	Testing what a component shows
MSW	Fake the network layer	Testing code that fetches data
interface Props	Type component inputs	Writing components in TypeScript
useState<T> + events	Type state, events and refs	Inputs, forms and DOM refs in TS
build + VITE_ env	Optimised output and config	Preparing to go live
Vercel / Netlify	Host the dist folder	Putting the app on a public URL



Thank You

Questions about any topic? Let's discuss.

Auth · JWT · Roles · Vitest · RTL · MSW · TypeScript · Build · Deploy

YOU FINISHED THE SERIES — NOW BUILD SOMETHING REAL